arXiv:2409.03052v1 [cs.LG] 4 Sep 2024

An Introduction to Centralized Training for Decentralized Execution in Cooperative Multi-Agent Reinforcement Learning

Christopher Amato, Northeastern University

December 22, 2024

Contents

1 The cooperative MARL problem: The Dec-POMDP **3**

2 CTDE overview **6**

3 Value function factorization methods **6**
3.1 Background on value-based RL . . . 7
3.2 VDN . . . 8
3.3 QMIX . . . 9
3.4 QTRAN . . . 13
3.5 QPLEX . . . 14
3.6 The use of state in factorization methods . . . 16

4 Centralized critic methods **16**
4.1 Preliminaries . . . 17
4.2 A basic centralized critic approach . . . 17
4.3 MADDPG . . . 20
4.4 COMA . . . 21
4.5 MAPPO . . . 21
4.6 State-based critics . . . 23
4.7 Choosing different types of decentralized and centralized critics . . . 24
4.8 Methods that combine policy gradient and value factorization . . . 25
4.9 Other centralized critic methods . . . 25

5 Other forms of CTDE **25**
5.1 Adding centralized information to decentralized methods . . . 25
5.2 Decentralizing centralized solutions . . . 26
5.3 Topics not discussed . . . 27

6 Acknowledgements **27**

Multi-agent reinforcement learning (MARL) has exploded in popularity in recent years. Many approaches have been developed but they can be divided into three main types: centralized training and execution (CTE), centralized training for decentralized execution (CTDE), and Decentralized training and execution (DTE).

CTE methods assume centralization during training and execution (e.g., with fast, free and perfect communication) and have the most information during execution. That is, the actions of each agent can depend on the information from all agents. As a result, a simple form of CTE can be achieved by using a single-agent RL method with centralized action and observation spaces (maintaining a centralized action-observation history for the partially observable case). CTE methods can potentially outperform the decentralized execution methods (since they allow centralized control) but are less scalable as the (centralized) action and observation spaces scale exponentially with the number of agents. CTE is typically only used in the cooperative MARL case since centralized control implies coordination on what actions will be selected by each agent. CTDE methods are the most common as they can use centralized information during training but execute in a decentralized manner—using only information available to that agent during execution. CTDE is the only paradigm that requires a separate training phase where any available information (e.g., other agent policies, underlying states) can be used. As a result, they can be more scalable than CTE methods, do not require communication during execution, and can often perform well. CTDE fits most naturally with the cooperative case, but can be potentially applied in competitive or mixed settings depending on what information is assumed to be observed. Decentralized training and execution methods make the fewest assumptions and are often simple to implement. In fact, any single-agent RL method can be used for DTE by just letting each agent learn separately. Of course, there are pros and cons to such approaches [Amato, 2024]. It is worth noting that DTE is required if no centralized training phase is available (e.g., though a centralized simulator), requiring all agents to learn during online interactions without prior coordination. DTE methods can be applied in cooperative, competitive, or mixed cases.

MARL methods can be further broken up into value-based and policy gradient methods. Value-based methods (e.g., Q-learning) learn a value function and then choose actions based on those values. Policy gradient methods learn an explicit policy representation and attempt to improve the policy in the direction of the gradient. Both classes of methods are widely used in MARL.

This text is an introduction to CTDE MARL. It is meant to explain the setting, basic concepts, and common methods. It does not cover all work in CTDE MARL as the subarea is quite extensive. I have included work that I believe is important for understanding the main concepts in the subarea and apologize to those that I have omitted.

I will first give a brief description of the cooperative MARL problem in the form of the Dec-POMDP. Then, I present an overview of CTDE and the two main classes of CTDE methods: value function factorization methods and centralized critic actor-critic methods. Value function factorization methods include the well-known VDN [Sunehag et al., 2017], QMIX [Rashid et al., 2018], and QPLEX [Wang et al., 2021a] approaches, while centralized critic methods include MADDPG [Lowe et al., 2017], COMA [Foerster et al., 2018b], and MAPPO [Yu et al., 2022]. Finally, I discuss other forms of CTDE such as adding centralized information to decentralized (i.e., independent) learners (such as parameter sharing) and decentralizing centralized solutions.

The basics of reinforcement learning (in the single-agent setting) are not presented in this text. Anyone interested in RL should read the book by Sutton and Barto [2018]. Similarly, for a broader overview of MARL, the recent book by Albrecht, Christianos and Schäfer is recommended [Albrecht et al., 2024].

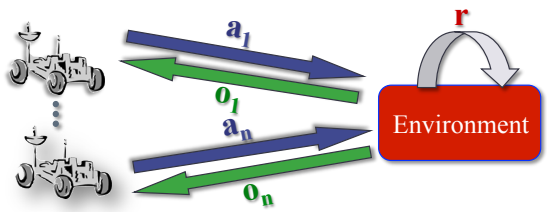


Figure 1: A depiction of cooperative MARL—a Dec-POMDP.

1 The cooperative MARL problem: The Dec-POMDP

The cooperative multi-agent reinforcement learning (MARL) problem can be represented as a Dec-POMDP [Oliehoek and Amato, 2016, Bernstein et al., 2002]. Dec-POMDPs generalize POMDPs [Kaelbling et al., 1998] (and MDPs [Puterman, 1994]) to the multi-agent, decentralized setting. As depicted in Figure 1, multiple agents operate under uncertainty based on partial views of the world, with execution unfolding over time. At each step, every agent chooses an action (in parallel) based purely on locally observable information, resulting in each agent obtaining an observation and the team obtaining a joint reward. The shared reward function makes the problem cooperative, but their local views mean that execution is decentralized.

Formally, a Dec-POMDP is defined by tuple $\langle \mathbb{I}, \mathbb{S}, \{\mathbb{A}_i\}, T, R, \{\mathbb{O}_i\}, O, \mathcal{H}, \gamma \rangle$. For simplicity, I define the finite version but it easily extends to the continuous case:

- $\mathbb{I}$ is a finite set of agents of size $|\mathbb{I}| = n$;
- $\mathbb{S}$ is a finite set of states with designated initial state distribution b_0;[1]
- $\mathbb{A}_i$ is a finite set of actions for each agent i with $\mathbb{A} = \times_i \mathbb{A}_i$ the set of joint actions;
- T is a state transition probability function, $T : \mathbb{S} \times \mathbb{A} \times \mathbb{S} \to [0, 1]$, that specifies the probability of transitioning from state $s \in \mathbb{S}$ to $s' \in \mathbb{S}$ when the actions $\mathbf{a} \in \mathbb{A}$ are taken by the agents (i.e., $T(s, \mathbf{a}, s') = \Pr(s'|\mathbf{a}, s)$);
- R is a reward function: $R : \mathbb{S} \times \mathbb{A} \to \mathbb{R}$, the immediate reward for being in state $s \in \mathbb{S}$ and taking the actions $\mathbf{a} \in \mathbb{A}$;
- $\mathbb{O}_i$ is a finite set of observations for each agent, i, with $\mathbb{O} = \times_i \mathbb{O}_i$ the set of joint observations;
- O is an observation probability function: $O : \mathbb{O} \times \mathbb{A} \times \mathbb{S} \to [0, 1]$, the probability of seeing observations $\mathbf{o} \in \mathbb{O}$ given actions $\mathbf{a} \in \mathbb{A}$ were taken and resulting in state $s' \in \mathbb{S}$ (i.e., $O(\mathbf{a}, s', \mathbf{o}) = \Pr(\mathbf{o}|\mathbf{a}, s')$);
- $\mathcal{H}$ is the number of steps until termination, called the horizon;
- and $\gamma \in [0, 1]$ is the discount factor.

[1] Some papers refer to other information such as joint observations or histories as 'state.' For clarity, I will only use this term to refer to the true underlying state.

I also include both the horizon and discount in the definition but, typically, only one is used. When $\mathcal{H}$ is finite, γ can be set to 1 and when $\mathcal{H} = \infty$, $\gamma \in [0, 1)$.[2]

A solution to a Dec-POMDP is a *joint policy*, denoted π—a set of policies, one for each agent, each of which is denoted π_i. Because the state is not directly observed, it is typically beneficial for each agent to remember a history of its observations (and potentially actions). Then, a *local policy*, π_i, for an agent is a mapping from local action-observation histories to actions or probability distributions over actions. A *local deterministic policy* for agent i, $\pi_i(h_i) = a_i$, maps $\mathbb{H}_i \to \mathbb{A}_i$, where $\mathbb{H}_i$ is the set of *local observation histories*, $h_i = \{a_{i,0}, o_{i,0}, \ldots, a_{i,t-1}, o_{i,t-1}\}$[3], by agent i up to the current time step, t. Note that histories have implicit time steps due to their length which we do not include in the notation (i.e., we always assume a history starts on the first time step and the last time step is defined by the number of action-observation pairs). We can denote the *joint histories* for all agents at a given time step as $\mathbf{h} = \langle h_1, \ldots, h_n \rangle$. A *joint deterministic policy* is denoted $\pi(\mathbf{h}) = \mathbf{a} = \langle \pi_1(h_1), \ldots, \pi_n(h_n) \rangle = \langle a_1, \ldots, a_n \rangle$. A *stochastic local policy* for agent i is $\pi_i(a_i|h_i)$, representing the probability of choosing action a_i in history h_i. A *joint stochastic policy* is denoted $\pi(\mathbf{a}|\mathbf{h}) = \prod_{i \in \mathbb{I}} \pi_i(a_i|h_i)$. Because one policy is generated for each agent and these policies depend only on local observations, they operate in a decentralized manner.

Many researchers just use observation histories (without including actions), which is sufficient for deterministic policies but may not be for stochastic policies [Oliehoek and Amato, 2016]. Deterministic policies will be used in the value-based methods in Section 3 while stochastic policies will be used in the policy gradient gradient methods in Section 4. There always exists an optimal deterministic joint policy in Dec-POMDPs [Oliehoek and Amato, 2016], but stochastic (or continuous) policies are needed for policy gradient methods.

The value of a joint policy, π, at joint history $\mathbf{h}$ can be defined for the case of discrete states and observations as

$$\mathbf{V}^{\pi}(\mathbf{h}) = \sum_{s} P(s|\mathbf{h}, b_0) \Big[R(s, \mathbf{a}) + \gamma \sum_{s'} P(s'|\mathbf{a}, s) \sum_{\mathbf{o}} P(\mathbf{o}|\mathbf{a}, s') \mathbf{V}^{\pi}(\mathbf{hao}) \Big] \Big|_{\mathbf{a}=\pi(\mathbf{h})} \tag{1}$$

where $P(s|\mathbf{h}, b_0)$ is the probability of state s after observing joint history $\mathbf{h}$ starting from state distribution b_0 and $\mathbf{a} = \pi(\mathbf{h})$ is the joint action taken at the joint history. Also, $\mathbf{hao}$ represents $\mathbf{h}'$, the joint history after taking joint action $\mathbf{a}$ in joint history $\mathbf{h}$ and observing joint observation $\mathbf{o}$. In the RL context, algorithms do not iterate over states and observations to explicitly calculate this expectation but approximate it through sampling. For the finite-horizon case, $\mathbf{V}^{\pi}(\mathbf{h}) = 0$ when the length of $\mathbf{h}$ equals the horizon $\mathcal{H}$, showing the value function includes the time step from the history.

We will often want to evaluate policies starting from the beginning—starting at the initial state distribution before any action or observation. Starting from this initial, null history we denote the value of a policy as $\mathbf{V}^{\pi}(\mathbf{h}_0)$. An *optimal joint policy* beginning at $\mathbf{h}_0$ is $\text{argmax}_{\pi} \mathbf{V}^{\pi}(\mathbf{h}_0)$, where the argmax here denotes enumeration over decentralized policies. The optimal joint policy is then the set of local policies for each agent that provides the highest value, which is denoted $\mathbf{V}^{*}$.

Reinforcement learning methods often use history-action values, $\mathbf{Q}(\mathbf{h}, \mathbf{a})$, rather than just history values $\mathbf{V}(\mathbf{h})$. $\mathbf{Q}^{\pi}(\mathbf{h}, \mathbf{a})$ is the value of choosing joint action $\mathbf{a}$ at joint history $\mathbf{h}$ and then

[2]The episodic case [Sutton and Barto, 2018] is typically indefinite-horizon with termination to a set of terminal states with probability 1 and the infinite-horizon case is sometimes called 'continuing.'

[3]Sometimes, $o_{i,0}$ is also included as an observation generated by the initial state distribution

continuing with policy π,

$$\mathbf{Q}^{\pi}(\mathbf{h}, \mathbf{a}) = \sum_{s} P(s|\mathbf{h}, b_0) \Big[R(\mathbf{a}, s) + \gamma \sum_{s'} P(s'|\mathbf{a}, s) \sum_{\mathbf{o}} P(\mathbf{o}|\mathbf{a}, s') \mathbf{Q}^{\pi}\big(\mathbf{hao}, \mathbf{a}'\big)|_{\mathbf{a}'=\pi(\mathbf{hao})} \Big], \tag{2}$$

while $\mathbf{Q}^{*}(\mathbf{h}, \mathbf{a})$ is the value of choosing action $\mathbf{a}$ at history $\mathbf{h}$ and then continuing with the optimal policy, π^{*}.

Policies that depend only on a single observation It is somewhat popular to define policies that depend only on a single observation rather than the whole observation history. That is, $\pi_i : \mathbb{O}_i \rightarrow \mathbb{A}_i$, rather than $\mathbb{H}_i \rightarrow \mathbb{A}_i$. This type of policy is often referred to as *reactive* or *Markov* since it just depends on (or reacts from) the past observation. These reactive policies are typically not desirable since they can be arbitrarily worse than policies that consider history information [Murphy, 2000] but can perform well or even be optimal in simpler subclasses of Dec-POMDPs [Goldman and Zilberstein, 2004]. In general, reactive policies reduce the complexity of finding a solution (since many fewer policies need to be considered) but only perform well in limited cases such as when the problem is not really partially observable or when the observation history isn't helpful for decision-making.

The fully observable case It is worth noting that if the cooperative problem is fully observable, it becomes much simpler. Specifically, a multi-agent MDP (MMDP) can be defined by the tuple $\langle \mathbb{I}, \mathbb{S}, \{\mathbb{A}_i\}, T, R, \mathcal{H}, \gamma \rangle$[Boutilier, 1996]. Each agent can observe the full state in this case. In the CTDE context, solving an MMDP can be done by standard MDP methods where the action space is the joint action space, $\mathbb{A} = \times_i \mathbb{A}_i$. The resulting policy, $\mathbb{S} \rightarrow \mathbb{A}$, can be trivially decentralized as $\mathbb{S} \rightarrow \mathbb{A}_i$ for each agent i. A number of methods have been developed for learning in MMDPs (and other multi-agent models) [Buşoniu et al., 2008].

Finding solutions in Dec-POMDPs is much more challenging. This can even be seen by the (finite-horizon) complexity for finite problems—optimally solving an MDP is P-complete (polynomial in the size of the system) [Papadimitriou and Tsitsiklis, 1987, Littman et al., 1995] and optimally solving a POMDP is PSPACE (essentially exponential in the actions and observations), [Papadimitriou and Tsitsiklis, 1987], while optimally solving a Dec-POMDP is NEXP-complete (essentially doubly exponential) [Bernstein et al., 2002]. This can be intuitively understood by considering the simplest possible solution method for each class of problems. For an MDP, you could search all policies mapping states to (joint) actions: $|\mathbb{A}|^{|\mathbb{S}|}$ of them. For a POMDP, you need to consider histories so centralized control (mapping joint histories to joint actions) would result in searching over $|\mathbb{A}|^{|\mathbb{O}|^{\mathcal{H}}}$ possibilities.[4] In the Dec-POMDP case, each agent would have $|\mathbb{A}_i|^{|\mathbb{O}_i|^{\mathcal{H}}}$ possible policies, so the possible number of possible joint policies would be $|\mathbb{A}_i|^{|\mathbb{O}_i|^{\mathcal{H}|\mathbb{I}|}}$ if all agents have the same number of actions and observations as agent i. Of course, state-of-the-art methods are more sophisticated than this and the complexity is for the worst case. Most real-world problems are intractable to solve optimally anyway. Regardless, these numbers make it clear that searching the joint space of history-based policies is enormous and the resulting partially observable problem is much more challenging than the fully observable case.

[4]Since we are assuming deterministic policies, the actions are fixed but we would still have to search over all possible observation histories up to the given horizon, resulting in $\mathbb{O}^{\mathcal{H}}$ possible histories.

2 CTDE overview

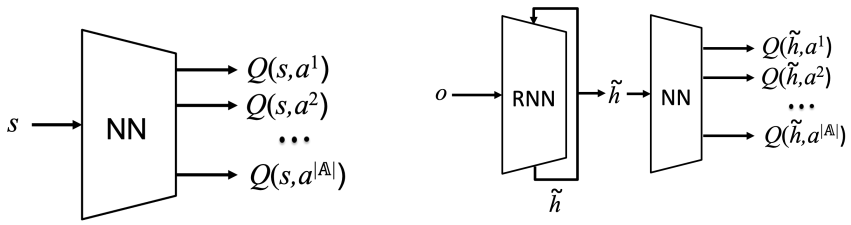
The idea of centralized training for decentralized execution is quite vague. The general idea of CTDE the Dec-POMDP case originated (with the Dec-POMDP itself) in the planning literature [Bernstein et al., 2002], where *planning* would be centralized but execution was decentralized. This idea goes back further to *team* decision making more generally since it is natural to think of deriving a solution for the team as a whole and then assigning corresponding parts to the team members [Marschak, 1955, Radner, 1962, Ho, 1980]. This concept was carried over to the reinforcement learning case and has come to mean that there is an 'offline'[5] training phase where some amount of centralization is allowed (such as in [Kraemer and Banerjee, 2016]). After this centralized training phase is complete, agents must then act in a decentralized manner based on their own histories (as given by the Dec-POMDP definition above). Any information that would not be available during decentralized execution can be used in the centralized training phase ranging from neural network parameters, policies of the other agents, a joint value estimate, etc. Nevertheless, CTDE has never been formally defined (to the best of my knowledge) and I will not do so here. I will take a wide view of CTDE and consider it to include any centralized information or coordination during learning—anything that is not available during decentralized execution.

CTDE methods for Dec-POMDPs vary widely. Some methods add centralized information to DTE methods. These approaches should be scalable but use limited centralized information. Other methods are at the other extreme, where they learn a centralized policy and then attempt to decentralize it so it can be executed without the centralized information. Most CTDE methods fall somewhere in between these two ideas. The main class of value-based CTDE methods learns a set of decentralized Q-functions (one for each agent) by factoring a joint value function, as in value function factorization methods. The main class of policy gradient CTDE methods learns a centralized critic that approximates the joint value function for all agents and that centralized critic is used to update a set of decentralized actors (one per agent). I discuss these ideas in more detail below.

3 Value function factorization methods

There is a long history of learning factored value functions in multi-agent reinforcement learning, but many previous methods were focused on more efficiently learning a joint value function (e.g., [Kok and Vlassis, 2006]). Other methods have also been developed that share Q-functions during training to improve coordination [Schneider et al., 1999]. Modern value function factorization methods learn Q-functions for each agent by combining them into a joint Q-function and calculating a loss based on the joint Q-function. In this way, the joint Q-function is factored into local, decentralized Q-values that can be used during decentralized execution. This idea is appealing because a joint Q-function can be learned to approximate the one in Equation 2 but agents can choose actions based on their decentralized Q-values. Additional details and the most popular methods are below.

[5] Offline does not refer to having a fixed training set [Levine et al., 2020] but rather in a different setting than what is used for execution (which would be considered online).



(a) DQN (b) DRQN

Figure 2: DQN and DRQN diagrams.

3.1 Background on value-based RL

I will first provide background on the value-based methods that the value function factorization methods build upon. Therefore, I will first introduce Deep Q-Networks (DQN), and the extension to partial observability, DRQN.

Deep Q-networks (DQN) [Mnih et al., 2015] is an extension of Q-learning [Watkins and Dayan, 1992] to include a neural net as a function approximator. Since DQN was designed for the fully observable (MDP) case, it learns $Q_\theta(s, a)$, parameterized with θ (i.e., θ represents the parameters of the neural network), by minimizing the loss:

$$\mathcal{L}(\theta) = \mathbb{E}_{<s,a,r,s'>\sim\mathcal{D}}\Big[\big(y - Q_\theta(s,a)\big)^2\Big], \text{where} \quad y = r + \gamma \max_{a'} Q_{\theta^-}(s', a') \tag{3}$$

which is just the squared TD error—the difference between the current estimated value, $Q_\theta(s, a)$, and the new value gotten from adding the newly seen reward to the previous Q-estimate at the next state, $Q_{\theta^-}(s', a')$. Because learning neural networks can be unstable, a separate target action-value function Q_{θ^-} and an experience replay buffer $\mathcal{D}$ [Lin, 1992] are implemented to stabilize learning. The target network is an older version of the Q-estimator that is updated periodically with $\langle s, a, r, s'\rangle$ sequences stored in the experience replay buffer and single $\langle s, a, r, s'\rangle$ tuples are i.i.d. sampled for updates. As shown in Figure 2(a), the neural network (NN) outputs values for all actions ($a \in \mathbb{A}$) to make maximizing over all states possible with a single forward pass (rather than iterating through the actions).

Deep recurrent Q-networks (DRQN) [Hausknecht and Stone, 2015] extends DQN to handle partial observability, where some number of recurrent layers (e.g., LSTM [Hochreiter and Schmidhuber, 1997]) are included to maintain an internal hidden state which is an abstraction of the history (as shown as $\tilde{h}$ in Figure 2(b)). Because the problem is partially observable, o, a, r, o' sequences are stored in the replay buffer during execution. The update equation is very similar to that of DQN:

$$\mathcal{L}(\theta) = \mathbb{E}_{<h,a,r,o>\sim\mathcal{D}}\Big[\big(y - Q_\theta(h,a)\big)^2\Big], \text{where} \quad y = r + \gamma \max_{a'} Q_{\theta^-}(h', a') \tag{4}$$

but since the recurrent neural network (RNN) is used, the internal state of the RNN can be thought of as a history representation. RNNs are trained sequentially, updating the internal state at each time step. As a result, the figure shows only o as the input but this assumes the internal state $\tilde{h}$ has already been updated using the history up to this point h^{t-1}. Therefore, rather than just sampling single updates (o, a, r, o') i.i.d, like in DQN, histories are sampled from the buffer. The internal

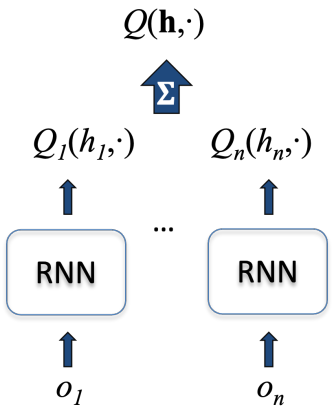


Figure 3: VDN architecture diagram

state can be updated incrementally and the Q-values learned starting from the first time step and going until the end of the history (i.e., the horizon or episode). An example of training using DRQN is shown in Algorithm 1. Technically, a whole history (e.g., episode) should be sampled from the replay buffer to train the recurrent network but it is common to use a fixed history length (e.g., sample o, a, o', r sequences of length 10). Also, as mentioned above, many implementations just use observation histories rather than full action-observation histories. Note that I will still write h rather than $\tilde{h}$ in the equations (e.g., $Q_\theta(h, a)$ above) to be more general. The resulting equations are not restricted to recurrent models (e.g., use a non-recurrent representation such as the full history or a Transformer-based representation [Esslinger et al., 2022, Ni et al., 2023]). With the popularity of DRQN, it has become common to add recurrent layers to standard (fully observable) deep reinforcement learning methods when solving partially observable problems.

3.2 VDN

Value decomposition networks (VDN) [Sunehag et al., 2017] began the popular trend in value decomposition methods for solving multi-agent reinforcement learning. The main contribution in VDN is a decomposition of the joint Q-function into additive Q-functions per agent.[6] This idea is relatively simple but very powerful. It allows training to take place in a centralized setting but agents can still execute in a decentralized manner because they learn individual Q-functions per agent, which can then be maxed over to select an action.

In particular, it assumes the following approximate factorization of the Q-function:

$$\mathbf{Q}(\mathbf{h}, \mathbf{a}) \approx \sum_{i \in \mathbb{I}}^{n} Q_i(h_i, a_i) \tag{5}$$

That is, the joint Q-function is approximated by a sum over each agent's individual Q-function. This is a form of factorization—the joint Q-function is factored as a sum of the local Q_i functions [Guestrin et al., 2001]. It is worth noting that each Q_i is technically a utility and not a value function since it is not estimating a particular expected return but can be any arbitrary value in the sum.

[6]The paper also uses dueling [Wang et al., 2016] and investigates forms of weight sharing and communication but we view these as orthogonal to the main value decomposition contribution.

The architecture diagram is given in Figure 3. Each agent learns an individual set of Q-values that depend only on local information. These Q-networks take the current observation as input to a recurrent network, which updates the history representation and then outputs Q-values for that agent. During training, these individual Q-values are summed to generate the joint Q-value which can then be used to calculate the loss like (a centralized form of) DRQN:

$$\mathcal{L}(\theta) = \mathbb{E}_{<\mathbf{h},\mathbf{a},r,\mathbf{o}>\sim\mathcal{D}}\Big[\big(y - \sum_{i}^{n} Q_i^{\theta}(h_i, a_i)\big)^2\Big], \text{where } \; y = r + \gamma \sum_{i}^{n} \max_{a_i'} Q_i^{\theta^-}(h_i', a_i') \qquad (6)$$

where h_i' is $h_i a_i o_i$ for h_i taken from $\mathbf{h}$, a_i from $\mathbf{a}$, and o_i from $\mathbf{o}$.

Since the loss uses the sum of the agent's Q-functions, the result is a set of Q-values whose sum approximates the joint Q-function. This approximation will be lossless in extreme cases (e.g., full independence) but it may be close or still permit the agents to select the best actions even if the approximation is poor. The approach is also scalable since the max used to create the target value y is not over all agent actions as in the centralized case ($\mathbb{A}$), but done separately for each agent in Equation 6.

A simple version of VDN is given in Algorithm 1. The approach is very similar to standard D(R)QN [Mnih et al., 2015, Hausknecht and Stone, 2015] as well as multi-agent DTE extensions [Omidshafiei et al., 2017, Tampuu et al., 2017]. The key difference is the calculation of the loss on Line 20, which approximates the joint Q-value by using the sum of the individual Q-values.

In more detail, the Q_i are estimated using RNNs for each agent parameterized by θ_i, along with using a target network, θ_i^-, and a replay buffer, $\mathcal{D}$. First, episodes are generated by interacting with the environment (e.g., ϵ-greedy exploration), which are added to the replay buffer and indexed by episode number and time step ($\mathcal{D}^e(t)$). This buffer typically has a fixed size and new data replaces old data when the buffer is full. Episodes can then be sampled from the replay buffer (either single episodes or as a minibatch of episodes). In order to have the correct internal state, RNNs are trained sequentially. Training is done from the beginning of the episode until the end, calculating the internal state and the loss at each step and updating each θ_i using gradient descent. The target networks θ_i^- are updated periodically (every C episodes in the code) by copying the parameters from θ.

While the algorithm is presented for the finite-horizon case (for simplicity), it can easily extended to the episodic and infinite-horizon cases by including terminal states or removing the loop over episodes.

After training, each agent keeps its own recurrent network, which can output the Q-values for that agent. The agent can then select an action by argmaxing over those Q-values: $\pi_i(h_i) = \text{argmax}_{a_i} Q_i(h_i, a_i)$.

VDN is often used a baseline but by using a simple sum to combine Q-functions, it can perform poorly compared to more sophisticated methods.

3.3 QMIX

QMIX [Rashid et al., 2018, 2020b] extends the value factorization idea of VDN to allow more general decompositions. In particular, instead of assuming the joint Q-function, $Q(\mathbf{h}, \mathbf{a})$, factors into a sum of local Q-values, $Q_i(h_i, a_i)$, QMIX assumes the joint Q-function is a monotonic function of the individual Q-functions. The sum used in VDN is also a monotonic function but QMIX allows more general (possibly nonlinear) monotonic functions to be learned.

Algorithm 1 A version of value decomposition networks (VDN) (finite-horizon)

set α, ϵ, and C (learning rate, exploration, and target update frequency)
Initialize network parameters θ_i for each Q_i (denoted Q_i^θ)
for all i, $\theta_i^- \leftarrow \theta_i$
$\mathcal{D} \leftarrow \emptyset$
$e \leftarrow 1$ {episode index}
for all episodes **do**
  for all $h_i \leftarrow \emptyset$ {initial history is empty}
  for $t = 1$ to $\mathcal{H}$ **do**
    for all i, take a_i at h_i from $Q_i^\theta(h_i, \cdot)$ with exploration (e.g., ϵ-greedy)
    See joint reward r_t, and observations $\mathbf{o}_t$
    append $\mathbf{a}, \mathbf{o}, r$ to $\mathcal{D}^e$
    for all $h_i \leftarrow h_i a_i o_i$ {update RNN state of the network}
  end for
  sample an episode from $\mathcal{D}$
  for $t = 1$ to $\mathcal{H}$ **do**
    for all i, $h_i \leftarrow \emptyset$
    $\mathbf{a}, \mathbf{o}, r \leftarrow \mathcal{D}^e(t)$
    for all i, $h_i' \leftarrow h_i a_i o_i$
    $y = r + \gamma \sum_i \max_{a_i'} Q_i^{\theta^-}(h_i', a_i')$
    for all i, do gradient descent on θ_i with learning rate α and loss $\left(y - \sum_i Q_i^\theta(h_i, a_i)\right)^2$
    for all i, $h_i \leftarrow h_i'$
  end for
  if $e \mod C = 0$ **then**
    for all i, $\theta_i^- \leftarrow \theta_i$
  end if
  $e \leftarrow e + 1$
end for
return all Q_i

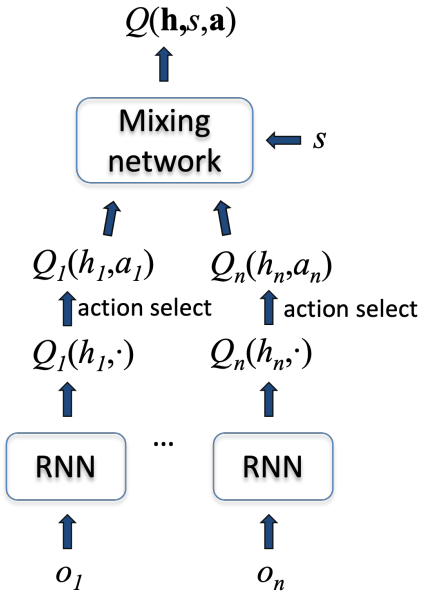


Figure 4: QMIX diagram

Specifically, QMIX assumes the following approximate factorization of the Q-function:

$$\mathbf{Q}(\mathbf{h}, \mathbf{a}) \approx f_{mono}(Q_i(h_1, a_1), \ldots, Q_n(h_n, a_n)) \tag{7}$$

This monotonic assumption means the argmax over the local Q-functions is also an argmax over the joint Q-function, This property allows agents to choose actions from their local Q-functions instead of having to choose them from a joint Q-function—ensuring the choice would be the same in both cases. Like in VDN, it also makes the training more efficient since calculating the argmax in the update Equation 9 is linear in the number of agents rather than exponential.

Son et al. [2019] formalized this property as the Individual-Global-Max (IGM) principal. IGM states that the argmax over the joint Q-function is the same as argmaxing over each agent's individual Q-function. The definition below is for a particular history but it should ideally hold for all histories (or at least the ones visited by the policy).

Definition 1 (Individual-Global-Max (IGM) Son et al. [2019]) *For a joint action-value function* $\mathbf{Q}(\mathbf{h}, \mathbf{a})$*, where* $\mathbf{h} = \langle h_1, \ldots, h_n \rangle$ *is a joint action-observation history, if there exist individual functions* $[Q_i]$*, such that:*

$$\underset{\mathbf{a}}{\operatorname{argmax}}\, \mathbf{Q}(\mathbf{h}, \mathbf{a}) = \begin{pmatrix} \operatorname{argmax}_{a_1} Q_1(h_1, a_1) \\ \vdots \\ \operatorname{argmax}_{a_n} Q_n(h_n, a_n) \end{pmatrix}, \tag{8}$$

then, $[Q_i]$ *satisfy IGM for* Q *at* $\mathbf{h}$*.*

A simplified version of the QMIX architecture is shown in Figure 4. Like VDN, each agent has an RNN that takes in the current observation and can output Q-values for the updated history over all actions.[7] A particular action is then chosen (e.g., using ϵ-greedy action selection) using each Q_i

[7] QMIX typically also inputs the previous action but this is omitted here for clarity and consistency.

and each $Q_i(h_i, a_i)$ is fed into the mixing network. The mixing network is made to be monotonic by restricting the weights (but not the bias terms) to be non-negative. The mixing network can also receive the state to potentially improve performance. In practice, the state is given as input to hypernetworks [Ha et al., 2017] which generate the weights for the layers of the mixing network. Because the output depends on the state, it is $\mathbf{Q}(\mathbf{h}, s, \mathbf{a})$, and is often called $\mathbf{Q}_{tot}$.

The network is trained end-to-end, like VDN by calculating a loss between $\mathbf{Q}(\mathbf{h}, s, \mathbf{a})$ and the joint return. The corresponding loss is:

$$\mathcal{L}(\theta) = \mathbb{E}_{<\mathbf{h},s,\mathbf{a},r,\mathbf{o},s'>\sim\mathcal{D}}\Big[\big(y - \mathbf{Q}^{\theta}(\mathbf{h}, s, \mathbf{a})\big)^2\Big], \text{where } y = r + \gamma \mathbf{Q}^{\theta^-}(\mathbf{h}', s', \tilde{\mathbf{a}}'),$$
$$\text{and } \tilde{\mathbf{a}}' = \langle \underset{a'_1}{\text{argmax}}\, Q_1(h'_1, a'_1), \ldots, \underset{a'_n}{\text{argmax}}\, Q_n(h'_n, a'_n)\rangle \quad (9)$$

where the value on the next time step, $\mathbf{Q}^{\theta^-}(\mathbf{h}', s', \tilde{\mathbf{a}}')$, is gotten by inputting the argmax over each agent's local Q-value using $\mathbf{h}' = \langle h'_1, \ldots, h'_n\rangle$ as well as the corresponding state s' that is sampled from the buffer at $\mathbf{h}'$.

The mixing network is only needed during training. Agents can retain their RNNs (which are just DRQNs) for selecting actions during decentralized execution just like VDN.

Again, such a factorization may not be possible in all problems so QMIX may not be able to approximate the true joint Q-function in all problems accurately. In particular, as discussed in the paper, this factorization should fail when agents' action choices depend on (at least some) other agents' actions at the same time step.

Weighted QMIX extends QMIX in an attempt to improve its expressiveness [Rashid et al., 2020a]. The main idea is to weigh the actions at different histories differently so the Q-values for some actions can be accurately represented but other action values can be less accurate. For instance, if you knew the optimal policy, you could set the weight highest for the optimal action in each history. You don't need to accurately represent the other action values but you do need to make sure they are lower than the optimal action (so the policy will choose the optimal action!).

Specifically, weighted QMIX uses weights $w(\mathbf{h}, \mathbf{a}) \in (0, 1]$ to adjust the importance of joint history-action pairs in the loss (as formalized below).[8] It turns out that there always exists some weighing that will allow the optimal policy to be recovered by independently argmaxing over each agent's Q_i. An idealized form of the algorithm could also converge in the fully observable case but this proof doesn't extend to the partially observable case or when approximations are used (i.e., the deep RL case).

The method has two main components, a QMIX network where the output is weighted, and an approximation of the optimal Q-value called $\hat{\mathbf{Q}}^*$. The first network takes the output of QMIX (as seen in Figure 4) and weighs it as seen in Equation 10. For easier differentiation with the other Q network, we call the output of the first (QMIX-style) network $\mathbf{Q}_{tot}$. This network is exactly the same as the one in QMIX but the output is weighted in the loss to prioritize different actions at different histories. The $\hat{\mathbf{Q}}^*$ network is similar to the one in QMIX but it does not limit the weights to be non-negative and doesn't use a hypernetwork (just directly inputting the state into the mixing network). These networks don't share parameters and are trained using the losses given in Equation

[8] The paper discusses much of the method and theory in terms of the fully observable case but we consider the partially observable case here. In the partially observable case, the weights can take the history or the history and the state: $w(h, s, a)$.

10. The same target (y) is used in both cases, which now uses the unconstrained output $\hat{\mathbf{Q}}^*$. The argmax is done the same way as in QMIX, where the actions are chosen using each agent's Q_i. As a result, the argmax is tractable but a potentially more accurate value of those actions can be provided from $\hat{\mathbf{Q}}^*$ rather than $\mathbf{Q}_{tot}$.

In particular, the losses uses are:

$$\begin{aligned}
&\mathcal{L}(\theta_{cent}) = \mathbb{E}_{<\mathbf{h},s,\mathbf{a},r,\mathbf{o},s'>\sim\mathcal{D}}\Big[\big(y - \hat{\mathbf{Q}}^*(\mathbf{h},s,\mathbf{a})\big)^2\Big],\\
&\mathcal{L}(\theta_{tot}) = \mathbb{E}_{<\mathbf{h},s,\mathbf{a},r,\mathbf{o},s'>\sim\mathcal{D}}\Big[w(\mathbf{h},\mathbf{a})\big(y - \mathbf{Q}_{tot}(\mathbf{h},s,\mathbf{a})\big)^2\Big],\\
&\text{where } y = r + \gamma\hat{\mathbf{Q}}^*(\mathbf{h}',s',\tilde{\mathbf{a}}'), \text{ and } \tilde{\mathbf{a}}' = \langle \underset{a'_1}{\text{argmax}}\, Q_1(h'_1,a'_1),\ldots,\underset{a'_n}{\text{argmax}}\, Q_n(h'_n,a'_n)\rangle
\end{aligned} \tag{10}$$

Two different weighting functions are considered. Centrally-Weighted QMIX (CW) uses:

$$w(\mathbf{h},\mathbf{a}) = \begin{cases} 1 & \text{if } y > \hat{\mathbf{Q}}^*(\mathbf{h},s,\tilde{\mathbf{a}}^*) \text{ or } \mathbf{a} = \tilde{\mathbf{a}}^* \\ \alpha & \text{otherwise} \end{cases}$$

which is an approximation of the optimal action $\tilde{\mathbf{a}}^*$ using the individual argmax as in Equation 10 and an approximation of the value function, $\hat{Q}^*$, using the unconstrained network. The weight is set to 1 when the action is already known to be optimal ($\mathbf{a} = \tilde{\mathbf{a}}^*$) or when the action has a higher value than the current best action. Optimistically-Weighted QMIX (OW) uses:

$$w(\mathbf{h},\mathbf{a}) = \begin{cases} 1 & \text{if } y > \mathbf{Q}_{tot}(\mathbf{h},s,\mathbf{a}) \\ \alpha & \text{otherwise} \end{cases}$$

which just sets the weight to 1 when the current estimate using $\mathbf{Q}_{tot}$ is less than the target value, which uses $\hat{\mathbf{Q}}^*$. This inequality suggests error in $\mathbf{Q}_{tot}$ and $\mathbf{a}$ could (optimistically) be optimal at $\mathbf{h}$. α is a hyperparamter that can be set during training.

After training, the Q_i from the constrained network (outputting $\mathbf{Q}_{tot}$) can be used for decentralized action selection (just like in QMIX). Therefore, $\hat{\mathbf{Q}}^*$ is only used to help guide the learning so better (hopefully, optimal) actions will have higher values in each agent's Q_i. While weighted QMIX can outperform QMIX in some cases, it isn't as widely used as more recent methods (e.g., QPLEX [Wang et al., 2021a] and MAPPO [Yu et al., 2022]).

3.4 QTRAN

QTRAN [Son et al., 2019] provides a different way to factor the joint Q-function into individual Q-functions. The idea generalizes VDN but the intuition is that the sum does not have to equal the true Q-function but some *transformed* Q-function $\mathbf{Q}'$. A separate V term[9] can then be used to account for the error between the transformed $\mathbf{Q}'$ and the true $\mathbf{Q}$. The approach cannot represent all IGM-able functions but it is more general than VDN.[10] QPLEX (which I talk about below) is more general and the relationship to QMIX and weighted QMIX is unclear.

Specifically, QTRAN uses Equation 11 as the basis for their architecture and losses.

[9] While the paper calls this a state value, the value takes histories as input, not states.

[10] While some have taken the text of the paper to show that QTRAN can represent all IGM-able functions, the proofs only show that the method satisfies IGM (not that any IGM-able function can be represented by the method).

$$\sum_i Q_i(h_i, a_i) - \mathbf{Q}(\mathbf{h}, \mathbf{a}) + \mathbf{V}(\mathbf{h}) = \begin{cases} 0 & \mathbf{a} = \tilde{\mathbf{a}} \\ \geq 0 & \mathbf{a} \neq \tilde{\mathbf{a}} \end{cases} \tag{11}$$

where

$$\mathbf{V}(\mathbf{h}) = \max_{\mathbf{a}} \mathbf{Q}(\mathbf{h}, \mathbf{a}) - \sum_i Q_i(h_i, \tilde{a}_i),$$

$$\tilde{a}_i = \operatorname*{argmax}_{a_i} Q_i(h_i, a_i), \quad \text{and} \quad \tilde{\mathbf{a}} = \langle \tilde{a_1}, \ldots, \tilde{a_n} \rangle$$

Here, $\mathbf{V}(\mathbf{h})$ has a particular form where it is the difference between the (centralized) max over the joint Q-function, $\mathbf{Q}$, and the sum of the maxes over the individual Q-functions, Q_i. Equation 11 is constrained to be 0 when the actions are the argmax over the individual Q-functions, $\tilde{\mathbf{a}}$, and greater than or equal to 0 for other actions. If Equation 11 holds, the Q_i satisfy IGM for the $\mathbf{Q}$ but it isn't clear how general it is. That is, unlike QPLEX, the proof does not show that all functions that satisfy IGM can be represented in this form.

Instead of enforcing a network structure to get IGM like VDN and QMIX, QTRAN does so by using losses based on Equation 11. The architecture outputs $\mathbf{Q}'(\mathbf{h}, \mathbf{a}) = \sum_i Q_i(h_i, a_i)$, like VDN, as well as an unconstrained $\mathbf{Q}(\mathbf{h}, \mathbf{a})$ and $\mathbf{V}(\mathbf{h})$. As shown below, there is the standard TD loss for learning the true joint Q-function, $\mathbf{Q}$, a loss for learning the transformed Q-function, $\mathbf{Q}'$, and value offset, $\mathbf{V}$, for the given $\mathbf{Q}$ when the actions are currently the maximizing ones, $\tilde{\mathbf{a}}$, in $\mathcal{L}_{opt}(\theta)$ and for other actions in $\mathcal{L}_{nopt}(\theta)$. The notation $\bar{\mathbf{Q}}$ is used to note that $\mathbf{Q}$ isn't trained from $\mathcal{L}_{opt}(\theta)$ and $\mathcal{L}_{nopt}(\theta)$.

$$\begin{aligned}
&\mathcal{L}_{td}(\theta) = \mathbb{E}_{<\mathbf{h},\mathbf{a},r,\mathbf{o}>\sim\mathcal{D}}\Big[\big(y - \mathbf{Q}(\mathbf{h}, \mathbf{a})\big)^2\Big], \\
&\mathcal{L}_{opt(\theta)} = \mathbb{E}_{<\mathbf{h},\mathbf{a},r,\mathbf{o}>\sim\mathcal{D}}\Big[\big(\mathbf{Q}'(\mathbf{h}, \tilde{\mathbf{a}}) - \bar{\mathbf{Q}}(\mathbf{h}, \tilde{\mathbf{a}}) + \mathbf{V}(\mathbf{h})\big)^2\Big], \\
&\mathcal{L}_{nopt}(\theta) = \mathbb{E}_{<\mathbf{h},\mathbf{a},r,\mathbf{o}>\sim\mathcal{D}}\Big[\Big(\min\big[\mathbf{Q}'(\mathbf{h}, \mathbf{a}) - \bar{\mathbf{Q}}(\mathbf{h}, \mathbf{a}) + \mathbf{V}(\mathbf{h}), 0\big]\Big)^2\Big], \\
&\text{where } y = r + \gamma \mathbf{Q}_{\theta^-}(\mathbf{h}', \tilde{\mathbf{a}}'), \text{ and } \tilde{\mathbf{a}}' = \langle \operatorname*{argmax}_{a'_1} Q_1(h'_1, a'_1), \ldots, \operatorname*{argmax}_{a'_n} Q_n(h'_n, a'_n) \rangle
\end{aligned} \tag{12}$$

The losses above are used for the standard form of QTRAN called QTRAN-base. An alternate form, called QTRAN-alt, replaces $\mathcal{L}(\theta_{nopt})$ with the average counterfactual loss below, forcing the value to be 0 for some action rather than relying on the inequality constraint above.

$$\mathcal{L}(\theta_{nopt-min}) = \mathbb{E}_{<\mathbf{h},\mathbf{a},r,\mathbf{o}>\sim\mathcal{D}}\Big[\frac{1}{n}\sum_i^n \Big(\min \mathbf{Q}'(\mathbf{h}, a_i, \mathbf{a}_{-i}) - \bar{\mathbf{Q}}(\mathbf{h}, a_i, \mathbf{a}_{-i}) + \mathbf{V}(\mathbf{h})\Big)^2\Big],$$

Using this condition also satisfies IGM.

The versions of QTRAN (QTRAN-base and QTRAN-alt) can outperform VDN and QMIX on some domains but they are often outperformed by more recent methods.

3.5 QPLEX

QPLEX [Wang et al., 2021a] further extends the value factorization idea to (provably) include more general decentralized policies. In particular, QPLEX can potentially learn any set of Q-functions that satisfy the IGM principle.

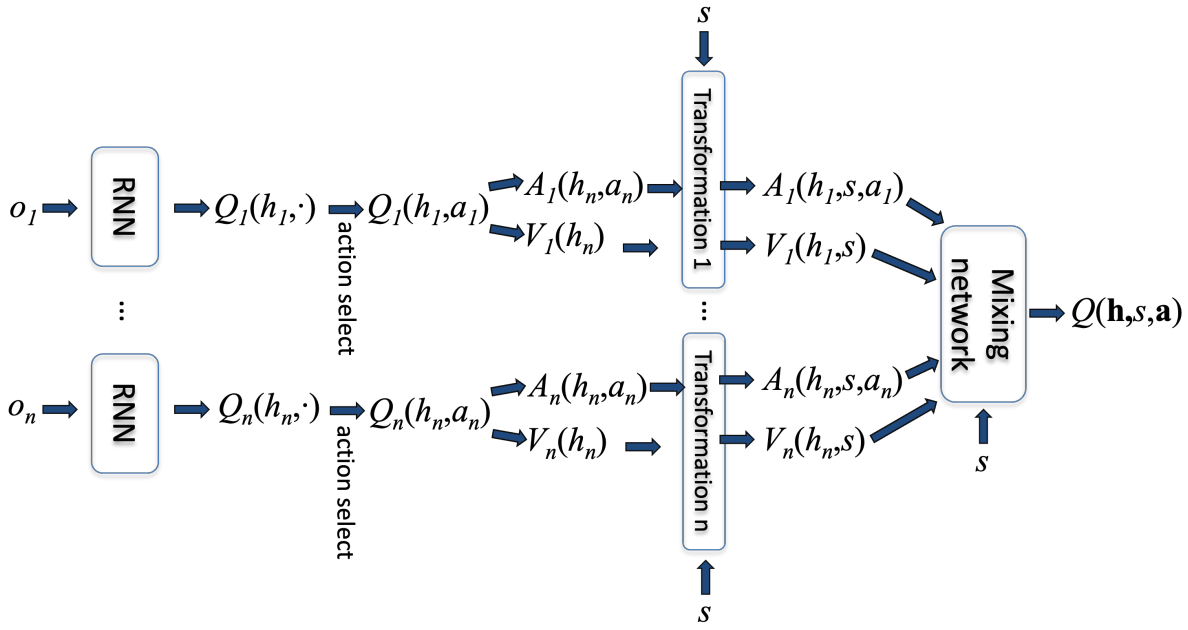


Figure 5: QPLEX diagram

First, QPLEX extends the IGM principle to an advantage-based case. They define joint values and advantages from the joint Q-function: $\mathbf{V}(\mathbf{h}) = \max_{\mathbf{a}} \mathbf{Q}(\mathbf{h}, \mathbf{a})$, and $\mathbf{A}(\mathbf{h}, \mathbf{a}) = \mathbf{Q}(\mathbf{h}, \mathbf{a}) - \mathbf{V}(\mathbf{h})$ as well as individual values and advantages from each agent's individual Q-function: $V_i(h_i) = \max_{a_i} Q_i(h_i, a_i)$, and $A_i(h_i, a_i) = Q_i(h_i, a_i) - V_i(h_i)$. Note that this notion of advantage is a bit different than the typical notion (such as [Wang et al., 2016]) since the values and advantages are generated from the Q-values, rather than the other way around. As a result, the advantages will have the property that they will be 0 for optimal actions and negative otherwise. That is, $A_i(h_i, a_i^*) = 0$ (since $Q_i(h_i, a_i^*) - \max_{a_i} Q_i(h_i, a_i) = 0$) and for some non-optimal action $a_i^\dagger$, $A_i(h_i, a_i^\dagger) < 0$ (since $Q_i(h_i, a_i^\dagger) < \max_{a_i} Q_i(h_i, a_i)$). The same holds true for joint advantages.

Advantage-based IGM extends IGM to the advantage case and states that the argmax over the joint advantage function is the same as argmaxing over each agent's individual advantage function:

Definition 2 (Advantage-based (IGM) Wang et al. [2021a]) *For a joint action-value function, if there exist individual functions $[Q_i]$, such that:*

$$\operatorname*{argmax}_{\mathbf{a}} \mathbf{A}(\mathbf{h}, \mathbf{a}) = \begin{pmatrix} \operatorname{argmax}_{a_1} A_1(h_1, a_1) \\ \vdots \\ \operatorname{argmax}_{a_n} A_n(h_n, a_n) \end{pmatrix}. \tag{13}$$

where $\mathbf{A}(\mathbf{h}, \mathbf{a})$ and A_i are defined above, then $[Q_i]$ satisfy advantage-based IGM for $\mathbf{Q}$ at $\mathbf{h}$.

That is, advantage-based IGM is equivalent to the Q-based IGM in Definition 1.

QPLEX uses advantage-based IGM to extend QMIX and QTRAN to represent the full IGM function class. The architecture is shown in Figure 5. Like QMIX (and VDN and QTRAN), QPLEX first takes as input each agent's observation o_i (and previous action a_i^{t-1} but that is not included here) and outputs the individual Q-values, $Q_i(h_i, a_i)$, for all a_i using a DRQN-style

network. After an action is selected (e.g., using ϵ-greedy exploration), the value and advantage are extracted from the Q-values for each agent as $V_i(h_i) = \max_{a_i} Q_i(h_i, a_i)$ and $A_i(h_i, a_i) = Q_i(h_i, a_i) - V_i(h_i)$. Next, the transformation networks generate $V_i(h_i, s)$ and $A_i(h_i, s, a_i)$ for each agent with the given $V_i(h_i)$ and $A_i(h_i, a_i)$ along with s as $V_i(h_i, s) = w_i(s)V_i(h_i) + b_i(s)$ and $A_i(h_i, s, a_i) = w_i(s)A_i(h_i, a_i)$. Finally, the joint Q-value is output based on the output of each agent's transformation network and the state as $\mathbf{Q}(\mathbf{h}, s, \mathbf{a}) = \sum_i V_i(h_i, s) + \lambda_i(s, \mathbf{a})A_i(h_i, s, a_i)$.[11] While there aren't constraints on V due to IGM (because it doesn't include the actions), QPLEX uses a sum to combine the local V's. All the weights, w_i and λ_i (but not necessarily the biases b_i) are positive to maintain monotonicity (like QMIX) and thus satisfy the advantage IGM principle. The architecture is trained using the RL loss, just like QMIX in Equation 9.

While positive weights are used (like in QMIX), there are separate weights for A and V, and the weights for V don't depend on the action. As a result, QPLEX is more general than previous methods with the capacity to represent all Q-functions that satisfy IGM. QPLEX also performs well, often outperforming other value factorization methods and it is currently one of the best-performing CTDE methods.

3.6 The use of state in factorization methods

Many of the current methods use inconsistent notation about the inclusion of state (except QMIX). As a result, the theory is often developed without considering the state input. It turns out that, unlike the policy gradient case discussed in Section 4.6, the using the state doesn't introduce bias and is thus theoretically sound [Marchesini et al., 2024]. The intuition is that the state is additional information in these cases. It doesn't replace the history, but potentially augments it (similar to a history-state critic in Section 4.6). Furthermore, actions are not chosen based on the state but only the local, history-dependent Q-values, Q_i. Nevertheless, in implementations of algorithms such as QPLEX and weighted QMIX, the weights only take the state as input and not the history. This replacement of history with state in these contexts limits the representational power of the weights in partially observable problems. As a result, using the state as input in QPLEX (instead of the joint history) prevents it from being able to represent the full IGM class of functions. It still may be beneficial to use the state instead of the joint history but exploring what additional information to use and in what way is an interesting open question.

4 Centralized critic methods

Concurrently, Multi-Agent Deep Deterministic Policy Gradient (MADDPG) [Lowe et al., 2017] and COunterfactual Multi-Agent policy gradient (COMA) [Foerster et al., 2018b] popularized the use of centralized critics in MARL. I will first discuss the general class of algorithms that MADDPG and COMA represent—multi-agent actor-critic methods with centralized critics—and then give the details of MADDPG and COMA. I will then describe the very popular PPO-based extension, MAPPO [Yu et al., 2022]. Finally, I will discuss the (incorrect) use of state in centralized critics as well as the theoretical and practical tradeoffs in the various types of critics.

[11] Attention is used to train the λ weights more efficiently.



Figure 6: Decentralized critics (a) vs. a centralized critic (b).

4.1 Preliminaries

Single-agent policy gradient and actor-critic methods have also been extended to the Dec-POMDP case. In the multi-agent case, there is one actor per agent and either one critic per agent (as shown in 6(a) and discussed in [Amato, 2024]) or a shared, centralized critic (as shown in 6(b) and discussed below). The centralized critic can be used during CTDE but then each agent can act in a decentralized manner by using its actor. The basic motivation is to leverage the centralized information that is available during training and potentially combat nonstationarity that can happen in decentralized training. All the policy gradient approaches can generally scale to larger action spaces and have stronger (local) convergence guarantees than the value-based counterparts above.

Policy gradient methods use continuous-action or stochastic policies. Unless stated otherwise, I will assume a stochastic policy for each agent parameterized by ψ_i, where $\pi_i^{\psi_i}(a_i|h_i)$ represents the probability agent i will choose action a_i given the history h_i and parameters, ψ_i, $\Pr(a_i|h_i, \psi_i)$. This is in contrast to the value-based methods in Section 3 where deterministic policies were generated based on the learned value functions. Like the value-based methods, the policy gradient methods also assume algorithms receive agent actions, $\mathbf{a}$, the resulting observations, $\mathbf{o}$, and the joint reward, r at each time step.

4.2 A basic centralized critic approach

In the simplest form, we can consider a class of centralized critic methods where the joint history critic, which we denote as $\hat{\mathbf{Q}}(\mathbf{h}, \mathbf{a})$, estimates $\mathbf{Q}^{\pi}(\mathbf{h}, \mathbf{a})$ with parameters θ. $\hat{\mathbf{Q}}(\mathbf{h}, \mathbf{a})$ is then used to update each decentralized policy π_i. We call this approach independent actor with centralized critic (IACC) and it is described in Algorithm 2. Learning is over a set of (on-policy) episodes and the histories are initialized to be empty. Agent i's history at time step t is denoted $h_{i,t}$, while the set of histories for all agents is denoted $\mathbf{h}_t$. All agents choose an initial action from their policies $a_{i,0} \sim \pi_i(a_i|h_{i,0})$ and then the learning takes place over a fixed horizon of $\mathcal{H}$. At each step, the agents take the corresponding current action $\mathbf{a}_t$, receive the joint reward r_t and see observations $\mathbf{o}_t$. The histories for the next step are updated to include the action taken and observation seen, $h_{i,t}a_{i,t}o_{i,t}$, and actions are sampled for the next step, $a_{i,t+1} \sim \pi_i(a_i|h_{i,t+1})$. Then, the TD error can be computed with the current Q-values, and then is used to update the actors and critic. Each agent's actor is updated using the TD error from the joint Q-function (moving in the direction that improves the joint Q-value). The centralized critic is updated using a standard on-policy gradient update. The action for the next step, $t+1$, becomes the action for the current step, t, and the process continues until the end of the horizon and for each episode. The algorithm is written for the finite-horizon case but it can be adapted to the episodic or infinite-horizon case by including terminal states or removing the loop over episodes. Note that the histories make the value functions based on a particular time step as the history length provides the time step.

While a sample is used in the algorithm, the full gradient associated with each actor in IACC is represented as:

$$\nabla_{\psi_i} J = \mathbb{E}_{<\mathbf{h},\mathbf{a}>\sim\mathcal{D}} \left[\mathbf{Q}^{\pi}(\mathbf{h}, \mathbf{a}) \nabla_{\psi_i} \log \pi_i(a_i|h_i) \right] ,^{12} \tag{14}$$

where the approximation of the joint Q-function, $\hat{\mathbf{Q}}(\mathbf{h}, \mathbf{a})$, is used to update the policy parameters of agent i's actor, $\pi_i(a_i|h_i)$. Histories (i.e., observations) and actions are sampled according to the on-policy (discounted) visitation probability.

The objective, J, is to maximize expected (discounted) return starting from the initial state distribution b_0 as discussed in Section 1.

The critic is updated using the on-policy loss:

$$\mathcal{L}(\theta) = \mathbb{E}_{<\mathbf{h},\mathbf{a},r,\mathbf{h}'>\sim\mathcal{D}} \Big[\big(y - \hat{\mathbf{Q}}(\mathbf{h}, \mathbf{a}) \big)^2 \Big], \text{ where } \; y = r + \gamma \hat{\mathbf{Q}}(\mathbf{h}', \mathbf{a}')$$

Algorithm 2 Independent Actor Centralized Critic (IACC) (finite-horizon)

Initialize individual actor models $\pi_i(a_i|h_i)$, parameterized by ψ_i
Initialize centralized critic model $\hat{\mathbf{Q}}(\mathbf{h}, \mathbf{a})$, parameterized by θ
for all episodes **do**
  $h_{i,0} \leftarrow \emptyset$ {Empty initial history}
  Denote $\mathbf{h}_t$ as $\langle h_{1,0}, \ldots, h_{n,0} \rangle$ {Notation for joint variables}
  for all i, choose $a_{i,0}$ at $h_{i,0}$ from $\pi_i(a_i|h_{i,0})$
  Store $\mathbf{a}_t$ as $\langle a_{1,0}, \ldots, a_{n,0} \rangle$
  for $t = 0$ to $\mathcal{H} - 1$ **do**
    Take joint action $\mathbf{a}_t$, see joint reward r_t, and observations $\mathbf{o}_t$
    for all i, $h_{i,t+1} \leftarrow h_{i,t} a_{i,t} o_{i,t}$ {Append new action and obs to previous history}
    for all i, choose $a_{i,t+1}$ at $h_{i,t+1}$ from $\pi_i(a_i|h_{i,t+1})$
    Store $\mathbf{a}_{t+1}$ as $\langle a_{1,t+1}, \ldots, a_{n,t+1} \rangle$
    $\delta_t \leftarrow r_t + \gamma \hat{\mathbf{Q}}(\mathbf{h}_{t+1}, \mathbf{a}_{t+1}) - \hat{\mathbf{Q}}(\mathbf{h}_t, \mathbf{a}_t)$ {Compute centralized TD error}
    Compute critic gradient estimate: $\delta_t \nabla_\theta \hat{\mathbf{Q}}(\mathbf{h}_t, \mathbf{a}_t)$
    Update critic parameters θ using gradient estimate (e.g., $\theta \leftarrow \theta + \beta \delta_t \nabla_\theta \hat{\mathbf{Q}}(\mathbf{h}_t, \mathbf{a}_t)$ for learning rate β)
    for each agent i **do**
      Compute actor gradient estimate: $\gamma^t \hat{\mathbf{Q}}(\mathbf{h}_t, \mathbf{a}_t) \nabla_{\psi_i} \log \pi_i(a_{i,t}|h_{i,t})$
      Update actor parameters ψ_i using gradient estimate (e.g., $\psi_i \leftarrow \psi_i + \alpha \gamma^t \hat{\mathbf{Q}}(\mathbf{h}, \mathbf{a}) \nabla_{\psi_i} \log \pi_i(a_{i,t}|h_{i,t})$ for learning rate α)
    end for
  end for
end for

We can extend this algorithm to the more commonly used advantage actor-critic (A2C) case, as shown in Algorithm 3. The advantage is defined as $\mathbf{A}(\mathbf{h}, \mathbf{a}) = \mathbf{Q}(\mathbf{h}, \mathbf{a}) - \mathbf{V}(\mathbf{h})$, representing the difference between the Q-value for a given action and the V-value at that history. Because $\mathbf{V}(\mathbf{h})$ doesn't depend on the action, it is called a *baseline* and becomes a constant from the perspective of the policy gradient. As a result using $\mathbf{Q}(\mathbf{h}, \mathbf{a}) - \mathbf{V}(\mathbf{h})$ in place of $\mathbf{Q}(\mathbf{h}, \mathbf{a})$ doesn't change the

[12] For simplicity and to match common implementations, we ignore discounting but the true gradient would include it [Nota and Thomas, 2020].

convergence properties of the method. Nevertheless, using a baseline (e.g., subtracting $\mathbf{V}(\mathbf{h})$) can reduce the variance of the estimate and improve performance in practice. In the algorithm, $\mathbf{A}(\mathbf{h}, \mathbf{a})$ isn't explicitly stored since the TD error is used as an approximation for the advantage—$\hat{\mathbf{Q}}(\mathbf{h}, \mathbf{a})$ is approximated with $r_t + \gamma \hat{\mathbf{V}}(\mathbf{h}_{t+1})$, which is true in expectation. δ_t can be thought of as a sample of $\hat{\mathbf{Q}}(\mathbf{h}_t, \mathbf{a}_t) - \hat{\mathbf{V}}(\mathbf{h}_t) = \hat{\mathbf{A}}(\mathbf{h}_t, \mathbf{a}_t)$.

Algorithm 3 Independent Advantage Actor Centralized Critic (IA2CC) (finite-horizon)

Initialize individual actor models $\pi_i(a_i|h_i)$, parameterized by ψ_i
Initialize centralized critic model $\hat{\mathbf{V}}(\mathbf{h})$, parameterized by θ
for all episodes **do**
  for all i, $h_{i,0} \leftarrow \emptyset$ {Empty initial history}
  Denote $\mathbf{h}_t$ as $\langle h_{1,0}, \ldots, h_{n,0} \rangle$ {Notation for joint variables}
  for all i, choose $a_{i,0}$ at $h_{i,0}$ from $\pi_i(a_i|h_{i,0})$
  Store $\mathbf{a}_t$ as $\langle a_{1,0}, \ldots, a_{n,0} \rangle$
  for $t = 0$ to $\mathcal{H} - 1$ **do**
    Take joint action $\mathbf{a}_t$, see joint reward r_t, and observations $\mathbf{o}_t$
    for all i, $h_{i,t+1} \leftarrow h_{i,t} a_{i,t} o_{i,t}$ {Append new action and obs to previous history}
    for all i, choose $a_{i,t+1}$ at $h_{i,t+1}$ from $\pi_i(a_i|h_{i,t+1})$
    Store $\mathbf{a}_{t+1}$ as $\langle a_{1,t+1}, \ldots, a_{n,t+1} \rangle$
    $\delta_t \leftarrow r_t + \gamma \hat{\mathbf{V}}(\mathbf{h}_{t+1}) - \hat{\mathbf{V}}(\mathbf{h}_t)$ {Compute centralized advantage estimates (TD error)}
    Compute critic gradient estimate: $\delta_t \nabla \hat{\mathbf{V}}(\mathbf{h}_t)$
    Update critic parameters θ using gradient estimate (e.g., $\theta \leftarrow \theta + \beta \gamma \delta_t \nabla \hat{\mathbf{V}}(\mathbf{h}_t)$)
    for each agent i **do**
      Compute actor gradient estimate: $\gamma^t \delta_t \nabla \log \pi_i(a_{i,t}|h_{i,t})$
      Update actor parameters ψ_i using gradient estimate (e.g., $\psi_i \leftarrow \psi_i + \alpha \gamma^t \delta_t \nabla \log \pi_i(a_{i,t}|h_{i,t})$)
    end for
  end for
end for

While the critic is called *centralized* because it uses centralized information, it estimates the *joint* value function of the decentralized policies. That is, the centralized critic estimates the value of the current set of decentralized policies (not a centralized policy) [Lyu et al., 2023]. This is the correct thing to do since the critic's job is to evaluate the policies, which are decentralized in this case.

With appropriate assumptions (e.g., on exploration, learning rate, and function approximation), IACC (and IA2CC) will converge to a local optimum [Peshkin et al., 2000, Lyu et al., 2023, 2024]. These results put policy gradient methods on solid theoretical ground and I discuss details of the various methods as well as some theoretical shortcomings.

This idea of learning a centralized critic to update decentralized actors is very general. It can be (and has been) used with different types of critics, actors, and updates. I present some of the most popular methods below.

4.3 MADDPG

Multi-Agent DDPG (MADDPG) [Lowe et al., 2017] considers the more general case of possibly competitive agents as well as continuous actions. To deal with the different reward functions of different agents in the competitive case, separate centralized critics are learned for each agent. As we are only concerned with the cooperative case, we assume a single shared critic among agents, do not have to learn policy models of the other agents (since these are assumed to be accessible in a cooperative CTDE setting), and do not consider ensembles of other agent policies to improve robustness. MADDPG is also an off-policy method, unlike the previous algorithms discussed, so it makes use of replay buffer similar to DQN-based approaches. Nevertheless, MADDPG for the cooperative case is very similar to Algorithm 2 with the main changes noted below.

To deal with continuous actions, MADDPG extends the continuous action single-agent deep actor-critic method DDPG [Lillicrap et al., 2016] to the multi-agent case. I denote the deterministic continuous-action policies for each agent as μ_i. In the cooperative case, MADDPG uses the following policy gradient:

$$\nabla_{\psi_i} J \doteq (1-\gamma)\mathbb{E}_{x,\mathbf{a}\sim\rho(x,\mathbf{a})}\left[\nabla_{\psi_i}\mu_i(o_i)\nabla_{\mathbf{a}}\mathbf{Q}^{\pi}(x,\mathbf{a})\mid_{a_i=\mu_i(o_i)}\right], \tag{15}$$

where x and $\mathbf{a}$ are sampled according to the (discounted) visitation probability with x discussed below and $\mathbf{a} = \langle a_1, \ldots, a_n\rangle$. Since we can can no longer sum over actions and weigh their value by their probability as is done in the stochastic policy case, we have to consider the change in the Q-value function evaluated at the chosen action a_i for the agent [Silver et al., 2014].

Note that MADDPG considers reactive policies that only map from the last observation to an action (as seen in μ_i). As noted in Section 1, the policy, μ_i, should choose actions based on histories, h_i, rather than single observations, o_i. The Q-value is from a centralized critic so the paper states x "could consist of the observations of all agents [...] however, we could also include additional state information if available." More generally, x should be history representations for each agent and we discuss the use of states in more detail in 4.6. These issues are easily fixable and result in the following policy gradient (we can incorporate the state information later):

$$\nabla_{\psi_i} J \doteq (1-\gamma)\mathbb{E}_{x,\mathbf{a}\sim\rho(\mathbf{h},\mathbf{a})}\left[\nabla_{\psi_i}\mu_i(h_i)\nabla_{\mathbf{a}}\mathbf{Q}^{\pi}(\mathbf{h},\mathbf{a})\mid_{a_i=\mu_i(h_i)}\right]. \tag{16}$$

Because MADDPG is off-policy, it maintains a replay buffer like DQN-based approaches along with 'target' copies of both the actor and critic networks, $\mu_{\psi_i^-}$ and Q_{θ^-}. The Q-update (for the history-based case) is then:

$$\mathcal{L}(\theta) = \mathbb{E}_{<\mathbf{h},\mathbf{a},r,\mathbf{h}'>\sim\mathcal{D}}\Big[\big(y - Q_\theta(\mathbf{h},\mathbf{a})\big)^2\Big], \text{ where } \; y = r + \gamma Q_{\theta^-}(\mathbf{h}',\mathbf{a}')\mid_{a_i=\mu^-(h_i)\;\forall i\in\mathbb{I}} \tag{17}$$

MADDPG executes a stochastic exploration policy (e.g., by adding Gaussian noise to the current deterministic policy) while estimating (and optimizing) the value of the deterministic policy. This can be seen because while the action, $\mathbf{a}$, is sampled from the (behavior policy) dataset, the next action, $\mathbf{a}'$ is selected by the target policy, μ^-.

Algorithm 2 can be updated to incorporate these changes. Since the continuous-action policy is deterministic, (Guassian) noise is added when selecting actions to aid in exploration. The approach is off-policy, so the experiences are first stored in a replay buffer (like DQN and other value-based methods in Section 3). Similarly, episodes are sampled from the replay buffer and target networks used (again, like DRQN-based approaches) for the actor and the critic.

MADDPG is no longer widely used but the ideas (such as the centralized critic) have been adopted extensively.

4.4 COMA

The main contributions of Counterfactual Multi-Agent Policy Gradients (COMA) were the introduction of the centralized critic along with a counterfactual baseline [Foerster et al., 2018b]. As mentioned above, baselines are common in policy gradient methods since they are high variance. The baseline value is subtracted from the Q-value and it can be anything that is not dependent on the agent's action.

In the case of COMA, the motivation for the baseline is not only variance reduction but also better credit assignment by subtracting off the perceived contribution to the Q-value from the other agents. Specifically, it marginalizes out the agent actions from the Q-function to get an estimate of what the (counterfactual) Q-function would be while holding the other agent actions fixed. The result is an agent-specific advantage function that subtracts the baseline for agent i from the joint Q-function:

$$A_i(\mathbf{h}, \mathbf{a}) = \mathbf{Q}(\mathbf{h}, \mathbf{a}) - \sum_{a'_i} \pi_i(a'_i|h_i)\mathbf{Q}(\mathbf{h}, a'_i, \mathbf{a}_{-i})$$

This baseline no longer depends on agent i's action (in expectation) so it will not bias the gradient. Note that the COMA paper uses a state-based critic and advantage, $\mathbf{Q}(s, \mathbf{a})$ and $A_i(s, \mathbf{a})$, but this is incorrect as I'll discuss in Section 4.6.

Rather than have separate baseline networks for each agent, there is a single centralized critic that takes in the other agent actions, $\mathbf{a}_{-i}$, the joint observation, $\mathbf{o}$, an agent id, i, the proposed action a_i, and policy probabilities $\pi_i(a'_i|h_i)$, and outputs the advantage for that agent $A_i(\mathbf{h}, \mathbf{a})$ using the equation above. This network can also output the joint Q-values for all agent i's actions, $Q(\mathbf{h}, \cdot, \mathbf{a}_{-i})$, given the other agent actions, $\mathbf{a}_{-i}$, the joint observation, $\mathbf{o}$, and an agent id, i. Both of these values are needed for the algorithm.

Then, the COMA algorithm can be an extension of Algorithm 2. The critic update can be calculated in the same way (using the Q-values from the network described above) but the agent-specific advantage is incorporated in the actor update. That is, during the actor update (starting on line 16), the agent-advantage $A_i(\mathbf{h}, \mathbf{a})$ is calculated for the given agent and A_i is used in the actor gradient estimate instead of $\hat{\mathbf{Q}}$.

While COMA has been very influential, it isn't widely used since newer methods tend to outperform it.

4.5 MAPPO

Multi-Agent PPO (MAPPO) extends PPO [Schulman et al., 2017] to the centralized critic MARL case [Yu et al., 2022]. The motivation behind PPO is to adjust the magnitude of the policy update to learn quickly without becoming unstable. PPO does this using a simple clipped loss that approximates the more complex trust region update [Schulman et al., 2015].

Like IA2CC (Algorithm 3), MAPPO uses an advantage-based update (but not the agent-specific counterfactual one used in COMA). In particular, instead of the loss being $\gamma^t \delta_t \log \pi_i(a_{i,t}|h_{i,t})$ as reflected on line 17, which is an approximation of $\gamma^t \mathbf{A}_t \log \pi_i(a_{i,t}|h_{i,t})$, the loss becomes:

$$\mathcal{L}_{clip}^{MAPPO}(\psi_i) = \min\Big(r_{\psi_i,i}\mathbf{A}, \text{clip}(r_{\psi_i,i}, 1-\epsilon, 1+\epsilon)\mathbf{A}\Big), \tag{18}$$

where $r_{\psi_i,i} = \frac{\pi_{\psi_i}(a_i|h_i)}{\pi_{\psi_{i,old}}(a_i|h_i)}$.[13] Maximizing $\frac{\pi_{\psi_i}(a_i|h_i)}{\pi_{\psi_{i,old}}(a_i|h_i)}\mathbf{A}$ seeks to maximally improve the new policy, π_{ψ_i}, compared to the old policy, $\pi_{\psi_{i,old}}$, by reweighing the advantage using an importance sampling ratio (considering the advantage was calculated using the old policy). It can be difficult to estimate this value from a small number of samples and it may result in too large of an update (resulting in parameters that perform worse). As a result, PPO also considered a term using a clipped ratio, limiting $r_{\psi_i,i}$ so it can't be too far above or below 1 (which would be the value when the new policy is equal to the old one). By minimizing over the unclipped and clipped values, PPO limits changes in the policy. Note the ratio is always positive but the advantage could be positive or negative. When the advantage is positive, the loss will be clipped if the ratio is too large, limiting how much more likely the action can be in the policy. Similarly, when the advantage is negative, the loss will be clipped when the ratio is near 0, limiting how much less likely the action can be in the policy. Like in IA2CC, $\mathbf{A}$ can still be approximated as $r_t + \gamma\hat{\mathbf{V}}(\mathbf{h}_{t+1}) - \hat{\mathbf{V}}(\mathbf{h}_t)$.

The critic loss for MAPPO also includes clipping and is given by:

$$\mathcal{L}^{MAPPO}(\theta) = \max\left[(\mathbf{V}(\mathbf{h}_t) - \hat{R}_t)^2, \left(\text{clip}(\mathbf{V}(\mathbf{h}), \mathbf{V}_{old}(\mathbf{h}) - \epsilon, \mathbf{V}_{old}(\mathbf{h}) + \epsilon) - \hat{R}_t\right)^2\right] \quad (19)$$

where $\hat{R}$ are Monte Carlo returns starting from the current history (i.e., $\hat{R}_t = \sum_{j=t}^{\mathcal{H}} \gamma r_j$) and $\mathbf{V}_{old}$ represents the value function from the previous time step but TD can also be used and value clipping does not have to be used.

All agents use the same centralized critic and parameter sharing can also used so all agents share the same actor network.

Independent PPO (IPPO) is a version of MAPPO where each agent uses its own local advantage rather than the joint advantage. As a result, the IPPO actor loss becomes:

$$\mathcal{L}^{IPPO}_{clip}(\psi_i) = \min\left(r_{\psi_i,i}A_i, \text{clip}(r_{\psi_i,i}, 1-\epsilon, 1+\epsilon)A_i\right), \quad (20)$$

with $\hat{A}_i = r_t + \gamma\hat{V}_i(h_{i,t+1}) - \hat{V}_i(h_{i,t})$. The IPPO critic loss becomes:

$$\mathcal{L}^{IPPO}(\theta) = \max\left[(V_i(h_{i,t})) - \hat{R}_t)^2, \left(\text{clip}(V_i(h_{i,t})), V_{i,old}(h_{i,t})) - \epsilon, V_{i,old}(h_{i,t})) + \epsilon) - \hat{R}_t\right)^2\right] \quad (21)$$

where $\hat{R}$ are Monte Carlo returns starting from the current *local* history, $V_i(h_{i,t})$. Parameter sharing is used with IPPO so all agents share the actor and critic network, making it a CTDE approach. As a result, there is only one actor and one critic, but it is updated using the data from all agents.

MAPPO and IPPO perform very well on the standard benchmark domains. Both algorithms typically perform similarly, but a version of MAPPO where the information in the critic was hand designed to only include features that are relevant to the task could outperform the standard versions of MAPPO and IPPO. This makes sense as the agents did not have to learn what information was necessary, which is difficult in high-dimensional settings.

[13] Here, I consider the stochastic (not continuous) action case and for simplicity do not include GAE, mini-batching, or a policy entropy term (which is common in policy gradient methods to improve exploration). Also, the paper uses a local advantage loss, A_i, but this value would not be different than $\mathbf{A}$ in a general Dec-POMDP if it is updated using $\mathbf{A}$ and r since these quantities would be the same for all agents. Finally, policies over single observations rather than histories are used, which as noted before is typically not sufficient for partially observable problems (general Dec-POMDPs) and state-based critics are used, which is incorrect as discussed below.

4.6 State-based critics

Critics that use state (and not history) are unsound in partially observable environments (i.e., general Dec-POMDPs). That is, they are incorrect and could be arbitrarily bad. They can work well in domains that are nearly fully observable (such as SMAC [Samvelyan et al., 2019]) and domains in which remembering history information is not helpful (observations are not very informative or the task is too hard to solve). This result has been shown theoretically and empirically [Lyu et al., 2022, 2023].

In particular, MADDPG [Lowe et al., 2017] and COMA [Foerster et al., 2018b] popularized the idea of using state-based critics using the intuition that this ground truth information that is available during training can help address partial observability. In truth, it is typically a bad idea to eliminate partial observability in the critic. A Dec-POMDP typically *is* partially observable and the actor is using partially observable information (i.e., histories). The state-based critic can be biased (i.e., incorrect) because there is a mismatch between the actor and the critic. The actor needs to use history information to choose actions. The state-based critic uses state values instead. As a result, the state values are averaged over the histories that are visited and can't possibly have the correct history values unless there is a one-to-one mapping from states to histories (which would happen in the fully observable case). That is, the critic loses the information about partial observability, which is necessary for the actor to make good choices.

For example, in the class Dec-Tiger problem [Nair et al., 2003, Oliehoek and Amato, 2016], there are only 2 states (the tiger being on the left or on the right) but noisy observations are received when an agent listens (an agent can also choose to open one of the two doors). A state-based critic would only have two values $\mathbf{V}(s^1)$ and $\mathbf{V}(s^2)$ but there are an exponential number of histories that depends on the horizon. As more information is gathered (i.e., the same observation about the tiger's location is heard multiple times) the value of the history, $\mathbf{V}(\mathbf{h})$, should increase (so $\mathbf{V}(\mathbf{h}_{tiger-left-10-times-in-a-row}) > \mathbf{V}(\mathbf{h}_0)$, the initial history). This can't be reflected in the state-based critic. Furthermore, information-gathering actions (i.e., listening in this tiger problem) won't improve the value according to the critic so they shouldn't be taken. As a result, the state-based critic values will be incorrect in the Dec-Tiger problem and the agents would learn a policy to open one of the doors and hope for a good outcome (assuming the tiger is randomly initialized).

The updates to Algorithm 3 for the state-based critic case are straightforward—$\hat{\mathbf{V}}(s)$ is used in place of $\hat{\mathbf{V}}(\mathbf{h})$. Then, the TD error calculation becomes $\delta_t \leftarrow r_t + \gamma\hat{\mathbf{V}}(s_{t+1}) - \hat{\mathbf{V}}(s_t)$, which is used in the actor and critic updates.

It turns out that history-state critics, those that take both the state and history as input ($\mathbf{Q}(\mathbf{h}, s, \mathbf{a})$ or $\mathbf{V}(\mathbf{h}, s)$ are unbiased and often perform the best [Lyu et al., 2022, 2023, Yu et al., 2022]. This result has been theoretically and empirically shown for the general single and multi-agent cases [Baisero and Amato, 2022, Lyu et al., 2022, 2023] as well as empirically with variants of MAPPO [Yu et al., 2022]. The intuition for correctness of the history-state critic is that no history information is lost (as it is in the simpler state-based critic). That is, the correct history value function can be (and is) recovered from the history-state value: $\mathbf{Q}^\pi(\mathbf{h}, \mathbf{a}) = \mathbb{E}_{s|\mathbf{h}}\left[\mathbf{Q}^\pi(\mathbf{h}, s, \mathbf{a})\right]$. The MAPPO paper also showed that if you can handcraft the features of the history-state critic to retain only the relevant information, the performance can be the highest since the agent doesn't need to learn which information is relevant and which is not. Unfortunately, determining which information is relevant and properly separating it can be difficult to do. The algorithmic updates for the history-state critic case are also straightforward—$\hat{\mathbf{V}}(\mathbf{h}, s)$ is used in place of $\hat{\mathbf{V}}(\mathbf{h})$. The TD error calculation becomes $\delta_t \leftarrow r_t + \gamma\hat{\mathbf{V}}(\mathbf{h}, s_{t+1}) - \hat{\mathbf{V}}(\mathbf{h}, s_t)$, which is used in the actor and critic

updates.

4.7 Choosing different types of decentralized and centralized critics

While CTDE methods are (by far) the most popular form of MARL, they do not always perform the best. In fact, decentralized training and execution methods (DTE) [Amato, 2024] are actually quite close to CTDE methods. This is because DTE methods typically make the concurrent learning assumption where all the agents learn using the same algorithm, making updates at the same time on the joint data. This equivalence is seen explicitly in the policy gradient case as the gradient of the joint update is the same as the decentralized gradient [Peshkin et al., 2000]. This phenomenon has been also been studied theoretically and empirically with modern actor-critic methods [Lyu et al., 2021, 2023]. It turns out that while centralized critic actor-critic methods are often assumed to be better than DTE actor-critic methods, they are theoretically the same (with mild assumptions) and often empirically similar (and sometimes worse). Too much centralized information can sometimes be overwhelming, harming scalability of centralized-critic methods and leading to higher variance [Yu et al., 2022, Lyu et al., 2023].

The theory assumes learned critics (or sufficient Monte Carlo estimates) but different types of critics may be easier to learn in different cases. As a result, it is not straightforward to determine when to use each critic in practice. Centralized critics often cause higher variance actor updates since information from the other agents needs to be marginalized out (e.g., through sampling) while information from the other agents is already removed from the decentralized critics. Centralized critics can also be more difficult to learn in problems with large numbers of agents or with large action or observation sets, making decentralized critics more scalable. Decentralized critics may be difficult to learn due to agents changing their policies (i.e., nonstationarity), making critic estimates stale. State-based critics are often the easiest to learn (because no history representation needs to be learned) but are biased in partially observable settings and will have high variance (again, because of the mismatch between the actor and the critic). In general, learning a history representation is difficult. Agents need to figure out what history information is relevant and what is not from a noisy and often sparse RL signal. Using a recurrent network (or even a transformer) is usually not sufficient for this task. History-state critics further increase the variance in the policy update (because the state information needs to also be marginalized out) but can often be easier to learn than centralized critics with only history information.[14] As a result the choice of critic is often a bias-variance tradeoff since, in practice, a centralized critic should have a lower bias than decentralized critics with more stable values that are more easily updated when policies change but higher variance because the policy updates need to be averaged over other agents.

There are many different ways of performing CTDE, and current work has only scratched the surface. Studying the differences and similarities between DTE and CTDE variants of algorithms seems like a promising direction for understanding current methods and developing improved approaches for both cases. Determining what centralized information to use and how to best use it is another key question. Lastly, to the best of my knowledge, there are no globally optimal model-free MARL methods for Dec-POMDPs. This is surprising since there are optimal *planning* methods where the model is assumed known [Oliehoek and Amato, 2016] and many globally optimal model-free methods for single-agent RL [Sutton and Barto, 2018]. Developing such methods (even for the tabular case) would be interesting and may lead to new, better methods that could be

[14] The exact reason why this is the case is unclear and a great topic for future research!

approximated in the deep case.

4.8 Methods that combine policy gradient and value factorization

Several methods combine using a centralized critic with value factorization. One notable method is FACtored Multi-Agent Centralized policy gradients (FACMAC) [Peng et al., 2021] which extends MADDPG to include a QMIX-style factored centralized critic, but since the local Q-values don't need to be used for action selection (the actor does that), the factorization can be nonmonotonic. Also, rather than sampling the actions of other agents from the off-policy dataset, as MADDPG does, actions for other agents are sampled from the current policies during the actor updates. Decomposed Off-Policy policy gradient (DOP) [Wang et al., 2021b] uses a factored centralized critic that is a weighted sum of the local Q-values and a multi-agent extension of tree backup [Precup et al., 2000] to more efficiently calculate off-policy updates. While a centralized critic is learned, each agent's local Q-value estimate is used to update its actor and with mild assumptions the method is shown to converge to a local optimum even with the simple critic.

4.9 Other centralized critic methods

Many other methods use centralized critics. For instance, Iqbal and Sha [2019] use attention for determining which other agent information to include in the centralized critic. Qatten [Yang et al., 2020] extends QMIX (and weighted QMIX) to add structure to the mixing network, where the structure is inspired by a linear approximation of the Q-function and one set of weights is leaned using attention. A wide range of other approaches have been developed but are not included in this introduction to the area.

5 Other forms of CTDE

While the methods above are the ones that are most widely used, there are several other forms of CTDE. I discuss common approaches and include a note on the many other topics that are not in this text in order to focus on core issues of CTDE.

5.1 Adding centralized information to decentralized methods

As noted in the introduction, decentralized training and execution (DTE) methods are independent learning methods [Claus and Boutilier, 1998] where each agent learns on its own using only its own information. More details about decentralized training methods can be found in a companion text [Amato, 2024]. DTE fit well when there is no offline training phase or when scalability is the key factor. DTE methods can be augmented with centralized information to potentially improve performance. These extensions are discussed below.

Parameter sharing Many cooperative MARL methods use parameter sharing. As noted in Section 4.5, the idea behind parameter (or weight) sharing is instead of each agent using a different network to estimate the value function or policy, all agents share the same networks. The data from each agent can be used to update a single value network for an algorithm such as DRQN, speeding up learning. Agents can still perform differently due to observing different histories and agent

indices can be added to increase specialization [Gupta et al., 2017, Foerster et al., 2016]. While parameter sharing is typically used with homogeneous agents (i.e., those with the same action and observation spaces), it can also be used with heterogeneous agents [Terry et al., 2020]. Since parameter sharing requires agents to share networks during training, it couldn't be used for online training in a decentralized fashion. Nevertheless, decentralized algorithms can be extended to use parameter sharing (such as the case with IPPO). Parameter-sharing implementations may be more scalable than other forms of CTDE, are often simple to implement, and can perform well [Gupta et al., 2017, Foerster et al., 2018b, Yu et al., 2022].

Alternating learning A number of methods allow agents to take turns learning. For instance, decentralized learning methods [Amato, 2024] can be used but all agents are fixed (not learning) except for one. The learning agent learns until convergence, generating a best-response to the other agent policies. If this process continues until no agents can further improve their polices, the result is a Nash equilibrium [Banerjee et al., 2012, Su et al., 2024]. In limited settings with additional strong assumptions (e.g., deterministic environments, full observability, additional coordination mechanisms to ensure coordinated policies), such methods can potentially converge to an optimal solutions [Lauer and Riedmiller, 2000, Jiang and Lu, 2023, 2022]. While these methods are sometimes called 'decentralized' since coordination and communication is need during learning that is not available during execution, I consider them to be CTDE.

Kuba et al. [2022] use sequential agent updates (i.e., one agent updates at a time while holding the others fixed) in Heterogeneous-Agent Trust Region Policy Optimisation (HATRPO) and Heterogeneous-Agent Proximal Policy Optimisation (HAPPO). HATRPO and HAPPO build off of TRPO [Schulman et al., 2015] and PPO [Schulman et al., 2017] and remove parameter sharing used in MAPPO [Yu et al., 2022] (leading to the heterogeneous-agent name). Using this sequential update scheme, they can theoretically prove monotonic improvement for the fully observable case and the approaches can work well in practice.

Addressing nonstationarity Decentralized learning methods face a nonstationary problem due to changing policies of other agents. Some methods try to address this challenge by directly modeling these changes. For example, Foerster et al. [2017] propose using importance sampling to correct for the probability differences and decay old data or simpler 'fingerprints' (e.g., episode number) to mark the data's age. Other methods try to model the other agent learning updates as well [Foerster et al., 2018a, Willi et al., 2022].

5.2 Decentralizing centralized solutions

Another potential approach is to learn a centralized solution during training and then (attempt to) decentralize it before execution. This is much harder than it seems. A centralized policy can map $\mathbb{H} \to \mathbb{A}$ without the constraint (in the stochastic case) that $\pi(\mathbf{a}|\mathbf{h}) = \prod_{i \in \mathbb{I}} \pi_i(a_i|h_i)$. That is, each agent's policy can depend on *other agent histories*: $\pi(\mathbf{a}|\mathbf{h}) = \prod_{i \in \mathbb{I}} \pi_i(a_i|\mathbf{h})$. The centralized policy class is much larger and richer than the decentralized policy class.

For example, just considering deterministic policies of horizon $\mathcal{H}$, there would be $|\mathbb{A}_i|^{|\mathbb{O}_i|^{\mathcal{H}|\mathbb{I}|}}$ possible decentralized policies (assuming all agents have the same size action and observation sets) vs. $|\mathbb{A}|^{\mathbb{O}^{\mathcal{H}}}$ possible centralized policies. Plugging in 4 actions per agent, 5 observations, a horizon

(history-length) of 10 and 4 agents gives $4^{5^{10^4}} = 1.024 \times 10^{43}$ vs. $(4^4)^{(5^4)^{10}} = 2.56 \times 10^{62520}$ policies. This is a massive difference.

As a result, most centralized policies will not be directly decentralizeable in the sense that they will be equivalent.[15] Policies can only be decentralized when they don't depend on other agent information but centralized agents that make joint action choices based on the information of all agent can often perform much better. In fact, this centralized partially observable case is called a multi-agent POMDP and can be solved using single-agent methods [Oliehoek and Amato, 2016].

As a result of these issues, approaches that decentralize centralized solutions are not common. Nonetheless, there are some instances. For example, Li et al. [2023] attempt to reconstruct the global information using a reconstruction loss, other methods attempt to mimic a centralized controller [Lin et al., 2022], and others allow communication during training but reduce or remove it during execution [Foerster et al., 2016, Wang et al., 2020]. FOP (for Factorizes the Optimal joint Policy) [Zhang et al., 2021] makes the assumption that centralized solutions are decentralizeable and then seeks to learn solutions using a max entropy formulation along with a value factorization scheme that is similar to methods in Section 3. While FOP (and other methods) could be applied in general Dec-POMDP settings, they are likely to perform poorly when the centralized solution is sufficiently different from the decentralized solution.

5.3 Topics not discussed

There are many other issues that are important to CTDE but are not discussed in this text, including exploration and communication as well as other topics such as hierarchical methods, role decomposition, ad-hoc (or zero-shot) coordination, and multi-task approaches. While these (and other) topics are very important, I do not include them for the sake of brevity and simplicity.

6 Acknowledgements

I thank Andrea Baisero, Shuo Liu, and the other members of my Lab for Learning and Planning in Robotics (LLRP) for reading my (very) rough drafts and providing comments that helped improve the document.

References

S. V. Albrecht, F. Christianos, and L. Schäfer. *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. `https://www.marl-book.com`.

C. Amato. An introduction to decentralized training and execution in cooperative multi-agent reinforcement learning. *arXiv preprint arXiv:2405.06161*, 2024.

A. Baisero and C. Amato. Unbiased asymmetric reinforcement learning under partial observability. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2022.

[15] Note that this is not true in the fully observable case as discussed in Section 1.

B. Banerjee, J. Lyle, L. Kraemer, and R. Yellamraju. Sample bounded distributed reinforcement learning for decentralized POMDPs. In *Proceedings of the National Conference on Artificial Intelligence*, 2012.

D. S. Bernstein, R. Givan, N. Immerman, and S. Zilberstein. The complexity of decentralized control of Markov decision processes. *Mathematics of Operations Research*, 27(4):819–840, 2002.

C. Boutilier. Planning, learning and coordination in multiagent decision processes. In *Proceedings of the 6th Conference on Theoretical Aspects of Rationality and Knowledge*, 1996.

L. Buşoniu, R. Babuška, and B. De Schutter. A comprehensive survey of multi-agent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews*, 38(2):156–172, Mar. 2008.

C. Claus and C. Boutilier. The dynamics of reinforcement learning in cooperative multiagent systems. In *Proceedings of the National Conference on Artificial Intelligence*, pages 746–752, 1998.

K. Esslinger, R. Platt, and C. Amato. Deep transformer Q-networks for partially observable reinforcement learning. *arXiv preprint arXiv:2206.01078*, 2022.

J. Foerster, I. A. Assael, N. De Freitas, and S. Whiteson. Learning to communicate with deep multi-agent reinforcement learning. *Advances in Neural Information Processing Systems*, 29, 2016.

J. Foerster, N. Nardelli, G. Farquhar, T. Afouras, P. H. Torr, P. Kohli, and S. Whiteson. Stabilising experience replay for deep multi-agent reinforcement learning. In *Proceedings of the International Conference on Machine Learning*, pages 1146–1155, 2017.

J. Foerster, R. Y. Chen, M. Al-Shedivat, S. Whiteson, P. Abbeel, and I. Mordatch. Learning with opponent-learning awareness. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2018a.

J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson. Counterfactual multi-agent policy gradients. In *Proceedings of the National Conference on Artificial Intelligence*, 2018b.

C. V. Goldman and S. Zilberstein. Decentralized control of cooperative systems: Categorization and complexity analysis. *Journal of AI Research*, 22:143–174, 2004.

C. Guestrin, D. Koller, and R. Parr. Multiagent Planning with Factored MDPs. In *Advances in Neural Information Processing Systems*, 2001.

J. K. Gupta, M. Egorov, and M. Kochenderfer. Cooperative multi-agent control using deep reinforcement learning. In *Adaptive and Learning Agents Workshop at AAMAS*, 2017.

D. Ha, A. M. Dai, and Q. V. Le. Hypernetworks. In *Proceedings of the International Conference on Learning Representations*, 2017.

M. Hausknecht and P. Stone. Deep recurrent Q-learning for partially observable MDPs. *arXiv preprint arXiv:1507.06527*, 2015.

Y.-C. Ho. Team decision theory and information structures. *Proceedings of the IEEE*, 68(6): 644–654, 1980.

S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

S. Iqbal and F. Sha. Actor-attention-critic for multi-agent reinforcement learning. In *International conference on machine learning*, 2019.

J. Jiang and Z. Lu. I2Q: A fully decentralized q-learning algorithm. In *Advances in Neural Information Processing Systems*, pages 20469–20481, 2022.

J. Jiang and Z. Lu. Best possible Q-learning. *arXiv preprint arXiv:2302.01188*, 2023.

L. P. Kaelbling, M. L. Littman, and A. R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.

J. R. Kok and N. Vlassis. Collaborative multiagent reinforcement learning by payoff propagation. *Journal of Machine Learning Research*, 7:1789–1828, 2006.

L. Kraemer and B. Banerjee. Multi-agent reinforcement learning as a rehearsal for decentralized planning. *Neurocomputing*, 190:82–94, 2016.

J. G. Kuba, R. Chen, M. Wen, Y. Wen, F. Sun, J. Wang, and Y. Yang. Trust region policy optimisation in multi-agent reinforcement learning. In *Proceedings of the International Conference on Learning Representations*, 2022.

M. Lauer and M. A. Riedmiller. An algorithm for distributed reinforcement learning in cooperative multi-agent systems. In *Proceedings of the International Conference on Machine Learning*, 2000.

S. Levine, A. Kumar, G. Tucker, and J. Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.

D. Li, Z. Xu, B. Zhang, G. Zhou, Z. Zhang, and G. Fan. From explicit communication to tacit cooperation: A novel paradigm for cooperative marl. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2023.

T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. In *Proceedings of the International Conference on Learning Representations*, 2016.

A. T. Lin, M. Debord, K. Estabridis, G. Hewer, G. Montufar, and S. Osher. Decentralized multi-agents by imitation of a centralized controller. In *Mathematical and Scientific Machine Learning*, pages 619–651, 2022.

L.-J. Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine learning*, 8(3-4):293–321, 1992.

M. L. Littman, T. L. Dean, and L. P. Kaelbling. On the complexity of solving Markov decision problems. In *Proceedings of Uncertainty in Artificial Intelligence*, 1995.

R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*, 2017.

X. Lyu, Y. Xiao, B. Daley, and C. Amato. Contrasting centralized and decentralized critics in multi-agent reinforcement learning. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2021.

X. Lyu, Y. Xiao, A. Baisero, and C. Amato. A deeper understanding of state-based critics in multi-agent reinforcement learning. In *Proceedings of the National Conference on Artificial Intelligence*, 2022.

X. Lyu, A. Baisero, Y. Xiao, B. Daley, and C. Amato. On centralized critics in multi-agent reinforcement learning. *Journal of AI Research*, 77:235–294, 2023.

X. Lyu, A. Baisero, Y. Xiao, B. Daley, and C. Amato. On centralized critics in multi-agent reinforcement learning (updated version). *arXiv preprint arXiv: 2408.14597*, 2024.

E. Marchesini, A. Baisero, R. Bathi, and C. Amato. On stateful value factorization in multi-agent reinforcement learning. *arXiv preprint arXiv: 2408.15381*, 2024.

J. Marschak. Elements for a theory of teams. *Management Science*, 1:127–137, 1955.

V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529, 2015.

K. P. Murphy. A survey of POMDP solution techniques. Technical report, University of British Columbia, 2000.

R. Nair, M. Tambe, M. Yokoo, D. V. Pynadath, and S. Marsella. Taming decentralized POMDPs: Towards efficient policy computation for multiagent settings. In *Proceedings of the International Joint Conference on Artificial Intelligence*, pages 705–711, 2003.

T. Ni, M. Ma, B. Eysenbach, and P.-L. Bacon. When do transformers shine in RL? decoupling memory from credit assignment. In *Advances in Neural Information Processing Systems*, 2023.

C. Nota and P. S. Thomas. Is the policy gradient a gradient? In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2020.

F. A. Oliehoek and C. Amato. *A Concise Introduction to Decentralized POMDPs*. Springer, 2016.

S. Omidshafiei, J. Pazis, C. Amato, J. P. How, and J. Vian. Deep decentralized multi-task multi-agent reinforcement learning under partial observability. In *Proceedings of the International Conference on Machine Learning*, 2017.

C. H. Papadimitriou and J. N. Tsitsiklis. The complexity of Markov decision processes. *Mathematics of operations research*, 12(3):441–450, 1987.

B. Peng, T. Rashid, C. Schroeder de Witt, P.-A. Kamienny, P. Torr, W. Böhmer, and S. Whiteson. Facmac: Factored multi-agent centralised policy gradients. *Advances in Neural Information Processing Systems*, 2021.

L. Peshkin, K.-E. Kim, N. Meuleau, and L. P. Kaelbling. Learning to cooperate via policy search. In *Proceedings of Uncertainty in Artificial Intelligence*, 2000.

D. Precup, R. S. Sutton, and S. P. Singh. Eligibility traces for off-policy policy evaluation. In *Proceedings of the International Conference on Machine Learning*, 2000.

M. L. Puterman. *Markov Decision Processes—Discrete Stochastic Dynamic Programming*. John Wiley & Sons, Inc., 1994.

R. Radner. Team decision problems. *Annals of Mathematical Statistics*, 33:857–881, 1962.

T. Rashid, M. Samvelyan, C. Schroeder, G. Farquhar, J. Foerster, and S. Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proceedings of the International Conference on Machine Learning*, 2018.

T. Rashid, G. Farquhar, B. Peng, and S. Whiteson. Weighted QMIX: Expanding monotonic value function factorisation for deep multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020a.

T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson. Monotonic value function factorisation for deep multi-agent reinforcement learning. *Journal of Machine Learning Research*, 21(1):7234–7284, 2020b.

M. Samvelyan, T. Rashid, C. S. de Witt, G. Farquhar, N. Nardelli, T. G. J. Rudner, C.-M. Hung, P. H. S. Torr, J. Foerster, and S. Whiteson. The StarCraft Multi-Agent Challenge. *arXiv preprint arXiv:1902.04043*, 2019.

J. Schneider, W.-K. Wong, A. Moore, and M. Riedmiller. Distributed value functions. In *Proceedings of the International Conference on Machine Learning*, 1999.

J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust region policy optimization. In *Proceedings of the International Conference on Machine Learning*, 2015.

J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller. Deterministic policy gradient algorithms. In *Proceedings of the International Conference on Machine Learning*, 2014.

K. Son, D. Kim, W. J. Kang, D. E. Hostallero, and Y. Yi. QTRAN: Learning to factorize with transformation for cooperative multi-agent reinforcement learning. In *Proceedings of the International Conference on Machine Learning*, 2019.

K. Su, S. Zhou, J. Jiang, C. Gan, X. Wang, and Z. Lu. MA2QL: A minimalist approach to fully decentralized multi-agent reinforcement learning. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2024.

P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, and T. Graepel. Value-decomposition networks for cooperative multi-agent learning. arXiv:1706.05296, 2017.

R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction (second edition)*. The MIT Press, 2018.

A. Tampuu, T. Matiisen, D. Kodelja, I. Kuzovkin, K. Korjus, J. Aru, J. Aru, and R. Vicente. Multiagent cooperation and competition with deep reinforcement learning. *PloS one*, 12(4), 2017.

J. K. Terry, N. Grammel, S. Son, B. Black, and A. Agrawal. Revisiting parameter sharing in multi-agent deep reinforcement learning. *arXiv preprint arXiv:2005.13625*, 2020.

J. Wang, Z. Ren, T. Liu, Y. Yu, and C. Zhang. QPLEX: Duplex dueling multi-agent Q-learning. In *Proceedings of the International Conference on Learning Representations*, 2021a.

T. Wang, J. Wang, C. Zheng, and C. Zhang. Learning nearly decomposable value functions via communication minimization. In *Proceedings of the International Conference on Learning Representations*, 2020.

Y. Wang, B. Han, T. Wang, H. Dong, and C. Zhang. DOP: Off-policy multi-agent decomposed policy gradients. In *Proceedings of the International Conference on Learning Representations*, 2021b.

Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the International Conference on Machine Learning*, 2016.

C. J. C. H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8(3):279–292, May 1992.

T. Willi, A. H. Letcher, J. Treutlein, and J. Foerster. COLA: Consistent learning with opponent-learning awareness. In *Proceedings of the International Conference on Machine Learning*, 2022.

Y. Yang, J. Hao, B. Liao, K. Shao, G. Chen, W. Liu, and H. Tang. Qatten: A general framework for cooperative multiagent reinforcement learning. *arXiv preprint arXiv:2002.03939*, 2020.

C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu. The surprising effectiveness of PPO in cooperative multi-agent games. *Advances in Neural Information Processing Systems*, 35, 2022.

T. Zhang, Y. Li, C. Wang, G. Xie, and Z. Lu. FOP: Factorizing optimal joint policy of maximum-entropy multi-agent reinforcement learning. In *Proceedings of the International Conference on Machine Learning*, 2021.